

LLM Code Smells: A Taxonomy and Detection Approach

Zacharie CHENAIL-LARCHER^{1*†}, Brahim MAHMOUDI^{1*†},
Naouel MOHA¹, Quentin STIÉVENART²,
Florent AVELLANEDA²

¹*École de technologie supérieure, Montréal, Québec, Canada.

²Université du Québec à Montréal, Montréal, Québec, Canada.

*Corresponding author(s). E-mail(s):

zacharie.chenail.larcher@gmail.com; brahim.mahmoudi.1@ens.etsmtl.ca;

Contributing authors: naouel.moha@etsmtl.ca;

stievenart.quentin@uqam.ca; avellaneda.florent@uqam.ca;

†The first two authors contributed equally to this work and share first authorship.

Abstract

Large Language Models (LLMs) are increasingly integrated into software systems for diverse purposes, due to their versatility, flexibility, and ability to simulate human reasoning to some extent. However, poor integration of LLM inference in source code can undermine software system quality. Therefore, inadequate LLM integration coding practices must be documented to help developers mitigate such issues. Following our earlier work on LLM code smells, this paper consolidates and refines the concept by presenting a self-contained taxonomy and a catalog of nine LLM code smells. We also create SpecDetect4LLM, a static source code analysis tool for their detection, and conduct extensive empirical evaluations of its detection effectiveness (precision and recall) as well as the prevalence of LLM code smells across 692 open-source software projects (171,194 source files). Our results show that LLM code smells affect 73.5% of the analyzed systems, with a detection precision of 91.3% and a recall of 71.8%.

Keywords: LLM, Code Smells, Software Quality, Static Analysis, AI, LLM-integrating Systems

1 Introduction

In recent years, Large Language Models (LLMs) have revolutionized the way information is processed and have gained increasing importance in everyday life. Their impact is such that the number of LLM-related publications has grown rapidly year after year, increasing from 503 in 2019 to 7109 in 2024 across 77 related conferences [1]. They are also being integrated into a growing number of software systems [2], due to their versatility, eloquence, flexibility, and their ability to simulate human reasoning to some extent.

However, LLMs are not always reliable [3], and their integration into software systems can introduce new risks and challenges due to their fundamentally different nature compared to traditional software components, notably their non-determinism, distinct hyperparameters, and interaction through natural language. Moreover, their performance and behaviour can vary significantly depending on how they are used [4]. To ensure the reliability, robustness, performance, and maintainability of LLM-integrating systems (systems that use LLMs as components, ranging from simple inference to extensive agentic logic), it is essential to properly integrate them, both at the architectural level and within the source code itself [5]. Therefore, as previously done for general [6] and machine learning-specific practices [7], establishing coding guidelines, including the specification of code smells, is essential for LLM-integrating systems.

While prior studies have defined taxonomies of general defects in LLM-integrating systems [2] and prompt-related defects [8], to our knowledge, there is no dedicated concept addressing code-specific poor practices for the integration of LLMs in software systems. To avoid ambiguity, we use the term LLM code smells to refer to poor coding practices in human-written source code that integrates or orchestrates LLM inference within software systems, rather than code smells in source code generated by LLMs, which belong to a distinct line of work on defects and smells in LLM-generated code [9]. This notion was first introduced in our previous ICSE 2026 New Ideas and Emerging Results (NIER) paper, *Specification and Detection of LLM Code Smells* [10], which provided an initial catalog of five LLM code smells, a first version of *SpecDetect4LLM*, and preliminary empirical results on detection precision and prevalence across 200 open-source projects. Our previous paper provides preliminary results, while the present paper constitutes the full study.

This paper substantially extends our earlier work in the following ways. First, we revisit, refine, and consolidate the concept of LLM code smells itself. Second, we expand the catalog from five to nine smells and organise it into a taxonomy. Third, we extend *SpecDetect4LLM* to cover the newly identified smells and provide a more detailed description of its static analysis scope and design choices. Fourth, we substantially enlarge the empirical evaluation by evaluating recall in addition to precision. Fifth, we expand the evaluation corpus from 200 to 692 open-source LLM-integrating systems, which allows a broader and more robust prevalence assessment. Accordingly, the present paper builds on the earlier NIER paper but substantially extends it in scope, methodological detail, and empirical validation.

The contributions of this journal paper are as follows:

1. We provide a self-contained taxonomy and catalog of nine LLM code smells.
2. We extend *SpecDetect4LLM* to cover the enlarged catalog and clarify its current static analysis scope. *SpecDetect4LLM* is a standalone static analysis tool for detecting LLM code smells. It is derived from the AI-specific code smells detection tool *SpecDetect4AI* [11].
3. We empirically evaluate the detection effectiveness of *SpecDetect4LLM* in terms of both precision and recall on a stratified random sample of 381 Python source code files drawn from a corpus of 171,196 files.
4. We conduct a large-scale empirical assessment of the prevalence of LLM code smells across 692 open-source LLM-integrating systems.

This paper is structured as follows:

- **Background (2)** : Presents essential concepts and definitions.
- **General Methodology (3)**: Describes the overall structure and methodology followed in this work.
- **Catalog Construction (4)** : Presents the research methodology leveraged to construct the LLM code smell catalog, including a systematic review of scientific literature and structured mining of grey and empirical literature.
- **LLM Code Smells (5)**: Introduces the taxonomy and the catalog.
- **Detection Approach with *SpecDetect4LLM* (6)**: Explains the design and detection workflow of *SpecDetect4LLM*.
- **Validation Design and Results (7)**: Empirically studies the detection correctness of *SpecDetect4LLM*.
- **Prevalence Study (7.10)**: Describes our empirical assessment of LLM code smells prevalence in open-source systems and the methodology followed.
- **Limitations and threats to validity (8)**: Documents the limitations and threats to validity that affect our work.
- **Related Work (9)**: Discusses the state of the art and related scientific work.
- **Conclusion (10)**: Summarizes the paper and concludes it.

2 Background

This section includes the definitions and concepts required to understand this paper, including the different types of LLMs, the concept of LLM code smells, and the software quality attributes affected by these smells.

2.1 Classes of LLMs and Related Concepts

Large Language Models (LLMs). LLMs are large-scale text generation models that are generally based on the transformer architecture [12] and generate one text token at a time during inference. Beyond these general-purpose language models, recent progress has led to specialised variants designed to extend the range of supported inputs and inference behaviours. In particular, two important classes are Vision-Language Models (VLMs) and Reasoning Language Models (RLMs).

Vision-Language Models (VLMs). VLMs, also referred to as Large Vision-Language Models (LVLMs), extend standard LLMs by incorporating visual tokens in addition to text tokens. This multimodal capability allows them to jointly process images and natural language, thereby enabling the association of visual and textual information for tasks such as visual understanding, captioning, and multimodal question answering [13]. In this sense, VLMs can be seen as a natural extension of standard LLMs toward multimodal inputs.

Reasoning Language Models (RLMs). While VLMs mainly extend LLMs along the input modality dimension, RLMs extend them along the inference dimension. RLMs, also referred to as Reasoning Models, are language models designed to solve tasks by generating intermediate reasoning steps before producing a final answer. Compared to general-purpose LLMs, RLMs can provide more accurate results on complex tasks [14], but they generally require longer processing time and higher token costs due to the additional intermediate steps.

Tokens. Tokens are the basic units into which AI model inputs and outputs are divided for processing. Depending on the format, tokens may represent different types of information, such as text or visual content. In this work, unless explicitly stated otherwise, **the term token refers to text tokens**. **Text tokens** correspond to discrete **textual units** used as the basic elements for text generation and LLM inference. In VLMs, **visual tokens** represent encoded visual information that enables image processing.

2.2 LLM Code Smells

Code smells refer to low-level coding practices that degrade software quality without necessarily representing explicit bugs or failures [6]. They are typically granular source-code patterns that negatively affect software quality and may indicate deeper design issues within a system. Over time, this concept has been extended through the development of catalogs and taxonomies tailored to specific application domains. Therefore, **LLM code smells** are code-level poor practices specific to the integration and use of LLM and VLM inference within software systems. While they do not always directly cause bugs or failures, they undermine key software quality attributes such as maintainability, reliability, performance, and robustness.

Following this perspective, **LLM code smells** are code-level poor practices specific to the integration and use of LLM and VLM inference within software systems rather than code smells in source code generated by LLMs [9]. The study of code smells in LLM generated code belongs to a separate scope of research, whereas our work focuses on the quality of software that integrates or coordinates LLM based functionality. Although LLM code smells do not always directly lead to failures, they may still negatively affect key software quality attributes, including maintainability, reliability, performance, and robustness.

2.3 Effects and Quality Attributes

The LLM code smells presented in this paper may affect any of four software quality attributes: robustness, performance, maintainability, and reliability [15–17]. A code

smell associated with a given attribute undermines it.

- **Robustness.** We use robustness to refer to the **operational resilience** and the ability of a software system to remain available in the presence errors, failures, unexpected conditions, or adverse situations, such as malicious user behaviors. It characterizes the system’s capacity to remain functional in faulty scenarios that could otherwise lead to service outages. LLM code smells that undermine robustness may increase the likelihood of runtime failures, system crashes, or loss of service.
- **Performance.** We use performance to refer to the ability of a software system to **execute its tasks with minimal computational overhead and resource costs**, including execution time, memory usage, computational resources, and monetary cost. LLM code smells that affect performance may introduce unnecessary latency, memory consumption, or monetary costs during system execution.
- **Maintainability.** Maintainability is the **capacity to preserve the integrity** of the software artifact during both development and execution. Systems with low maintainability may exhibit unnecessary coupling between components, poorly documented settings, or insufficient logging and execution traces. We use maintainability to relate to **reproducibility, portability, traceability, and observability** of LLM-integrating systems. It concerns artifacts such as source code, configuration files and logs.
- **Reliability.** We use reliability to refer to the **ability of a software system and its LLM inference to consistently produce correct and effective results over time**. It captures both the quality of the system’s outputs and the stability of its behavior. LLM code smells that undermine reliability may degrade the quality of inference results, introduce behavioral inconsistencies, or lead to silent and passive changes in system behavior over time. In contrast to maintainability, which concerns the ease of evolving and managing a system, reliability focuses on the correctness, consistency, and behavioral stability of the system during execution.

3 General Methodology

This study aims to propose a taxonomy and catalog of LLM code smells as theoretical guidelines to support the development of LLM-integrating systems, assess their prevalence in real systems to validate their legitimacy, and provide a detection tool to facilitate their practical use. As shown in Figure 1, this work follows an overall methodology composed of three main phases. This section provides an overview of each of these phases, which are described in much greater detail across their respective dedicated sections.

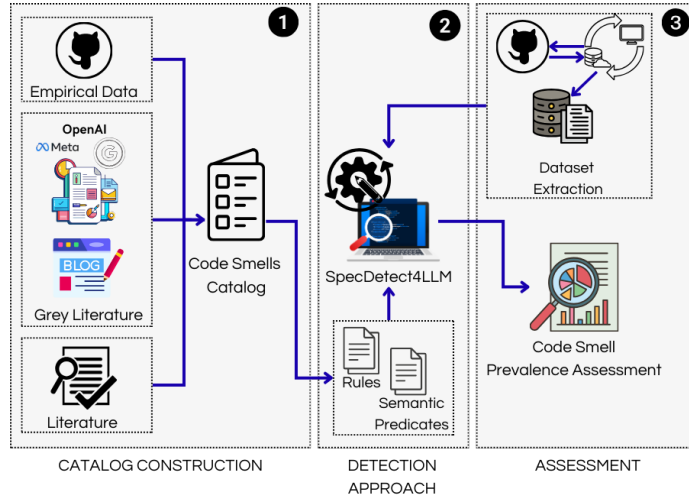


Fig. 1 Research design and workflow of the study.

3.1 Phase 1: Catalog Construction

Dedicated sections: *LLM Code Smells* (5) covers the taxonomy and presents the catalog, while *Catalog Construction* (4) presents in greater details the methodology of this phase in detail.

Goal: Consolidate evidence to produce a taxonomy and catalog of LLM code smells.

Inputs: Academic literature search queries, and research prompts for empirical and grey literature.

Outputs: A taxonomy and a catalog of LLM code smells.

Description: We examined three complementary families of sources, namely academic, grey, and empirical literature, following established guidance for evidence based synthesis in software engineering [18]. Candidate LLM code smells were retained when they were observable at the code level, supported by multiple sources, and associated with an accessible remediation strategy. They were then organised within the proposed taxonomy and documented using a fixed schema.

3.2 Phase 2: Detection Approach

Dedicated sections: *Detection Approach with SpecDetect4LLM* (6) describes the design of SpecDetect4LLM, while *Validation Design and Result* (7) evaluates it.

Goal: Develop a detection tool for LLM code smells based on static source code analysis, and validate it.

Inputs: The catalog of LLM code smells from the previous phase.

Outputs: *SpecDetect4LLM*, a static detection tool, an empirical evaluation of its effectiveness, a dataset of open-source LLM-integrating systems and a manually audited ground truth set comprising 381 files.

Description: We specified each catalogued smell as a declarative detection rule and implemented these rules using the domain specific language (DSL) of *SpecDetect4AI* [11]. We then applied *SpecDetect4LLM* to a corpus of 692 open source LLM-integrating systems and evaluated its effectiveness against manually audited annotations.

3.3 Phase 3: Prevalence Assessment

Dedicated sections: *Prevalence Study* (7.10)

Goal: Assess how prevalent LLM code smells and how they manifest in practice.

Inputs: The dataset of open-source LLM-integrating systems collected in the previous phase, and *SpecDetect4LLM*.

Outputs: Prevalence and occurrence metrics, statistics, and analysis of LLM code smells.

Description: Using *SpecDetect4LLM*, we conducted a prevalence study on a dataset of 692 LLM integrating systems comprising 171,194 files. We measured how much LLM code smells affect files and projects, both overall and for each smell individually, and then examined the main patterns observed in practice.

4 Catalog Construction

This section explains how we constructed the catalog of LLM code smells by combining three complementary strands of evidence, following established guidelines for systematic mapping studies in software engineering and multi-source triangulation [18].

First, we conducted a systematic review of research literature on defects, misuses, and bad practices that arise when integrating LLMs into software systems.

Second, we mined grey (blog posts, official documentation, etc.) and empirical (GitHub issues, open-source software projects, etc.) literature with GLiSE, a prompt-driven and machine-learning-based tool designed to automate the search, filtering, and extraction of relevant grey literature in software engineering [19] to capture practitioner experience and reports, as well as guidelines from sources such as provider documentation, engineering blogs, and Q&A platforms. These types of literature were included because real-world practices in software engineering are not always reflected in peer-reviewed research, and because LLM integration is still recent enough that provider documentation and community reports often represent the most accurate sources on how to integrate and use LLMs in practice.

For each candidate smell, we recorded which evidence types supported it and how many distinct sources contributed, which allows us to trace each smell back to the underlying the corpus.

4.1 Systematic review of research literature

We followed the updated PRISMA guidelines [20, 21] and Kitchenham et al. [22] guidelines to review and report our findings. We used three main phases: planning, conducting, and reporting the review. During the planning phase, we defined the objective of SLR and reviewed the protocol. The objective of this SLR is to answer

Table 1 Systematic Literature Review and Grey Literature Sources

Code Smell	Source	
	Literature	Grey and Empirical
No Structured Output (Section 5.2.1)	[54–59]	[60–115]
Unbounded Max Metrics (Section 5.2.2)	[55, 57, 58, 116–121]	[64, 65, 122–140]
LLM Temperature Not Explicitly Set (Section 5.2.3)	[55–57, 116, 119, 141–143]	[35, 64, 122, 123, 135–138, 140, 144–162, 164]
No Model Version Pinning (Section 5.2.4)	[116, 118, 165–172]	[122, 123, 145, 173, 174]
No System Message (Section 5.2.5)	[175–178]	[64, 145, 179–189]
Reasoning Effort Not Explicitly Set (RENES)	[119, 190–193]	
Raw Vision Payload (Section 5.2.7)	[194–196]	
Overspecified Sampling Parameters (Section 5.2.8)		[35, 135–137, 140, 157, 162, 197–204]
Anonymous Inference Call (Section 5.2.9)		[164, 205–215]
Others	[216–218]	[135–137, 139, 219–251]

* *Sources* - Literature: peer-reviewed literature; Grey: grey literature (official docs, tech reports, blogs); and Empirical: primary empirical data (e.g., commits, repos).

* *Others* - Refers to general sources and other relevant results that did not lead to new LLM code smells.

this question: which kinds of integration problems, misconfigurations, and code level bad practices are documented when LLMs are embedded into software systems, tools, or pipelines?

It consists of four steps:

- ❶ Definition of the research question
- ❷ Formulation of the search query
- ❸ Selection of the papers
- ❹ Snowballing

In the following subsections, we explain each step of our review protocol.

4.1.1 Definition of the research intent

This section answers the following question: *How can we derive a catalog of recurrent LLM code smells by consolidating evidence from scientific literature, grey literature, and practitioner artifacts?*

Answering this question yields a consolidated catalog of LLM integration defects that is grounded both in prior research and in the concrete practice of software teams. The resulting catalog provides a common vocabulary to describe recurrent integration problems, supports practitioners in recognizing these defects in their own systems, and serves as a foundation for automated detection approaches such as *SpecDetect4LLM*. In the remainder of this section, we explain how we operationalise the RQ into concrete search, screening, and coding procedures to capture evidence about LLM code smells across heterogeneous sources.

4.1.2 Formulation of the Search Query

We formulated our search query by applying the PICO (Population, Intervention, Comparison, Outcome) framework [23]. We followed the following steps:

- ❶ Obtaining the main terms from our research question (RQ1).
- ❷ Identifying the possible synonyms of the main terms.
- ❸ Applying the Boolean OR to combine possible synonyms of the main terms.
- ❹ Applying the Boolean AND to combine expressions in the previous step.

As a result of PICO framework, we formulated the following search query:

```
("large language model*" OR "LLM*" ) AND ("integrat*" OR "API*" OR "software system*" ) AND ("misuse*" OR "defect*" OR "bug*" OR "smell*" OR "pitfall*")
```

To obtain more comprehensive results, we used the asterisk (*) in search queries as a wildcard to match any sequence of characters.

4.1.3 Selection of the Papers

We applied the PRISMA steps to select the literature papers. The main steps include database identification, removal of duplicates, screening, eligibility assessment, two rounds of both backward and forward snowballing, and quality assessment of each study. Figure 2 summarizes those steps.

Databases Identification

We selected seven online digital libraries: ACM Digital Library, Compendex, IEEE Xplore, ScienceDirect, SpringerLink, Scopus, arXiv, and Wiley. These libraries are widely used for literature reviews in software engineering, as recommended by Dyba et al. [24].

We applied our search query to each of these digital libraries. However, some libraries impose restrictions when performing queries. For instance, ScienceDirect limits queries to a maximum of eight connectors, while the ACM Digital Library does not allow wildcards. We adjusted the search query to meet the specific requirements of each library.

Our search was confined to English-language, peer-reviewed scholarly articles published in journals, conferences, and workshops between 2017 and 2025. This time frame was chosen due to the increase in LLM-related literature since 2017 [25]. We initially retrieved a total of 2243 papers from seven libraries.

Duplicates Removal

Duplicates were identified based on an exact match of the study’s title, first author, and venue (conference or journal). This reduced the total from papers to 2117.

Screening

We defined inclusion and exclusion criteria and applied them to select relevant primary papers while excluding irrelevant ones.

Inclusion Criteria:

We considered the following inclusion criteria for paper selection:

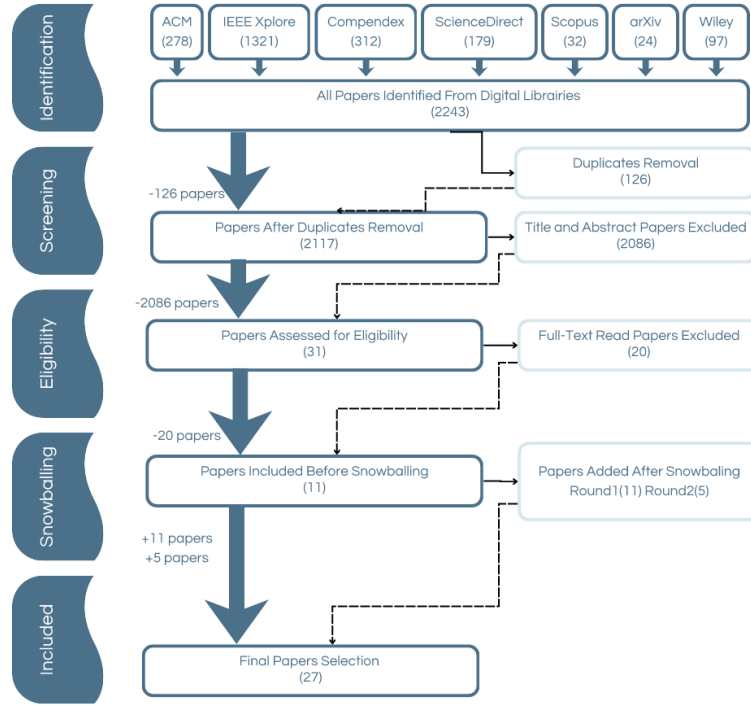


Fig. 2 PRISMA flow for paper selection. Values in parentheses indicate the number of papers at each stage.

- **IC1:** The study is written in English.
- **IC2:** The study is published between 2017 and 2025.
- **IC3:** The study presents an LLM based software system or an engineering practice involving LLM API or SDK integration.
- **IC4:** The study reports at least one concrete integration issue, misuse, defect, or failure related to LLM based components.
- **IC5:** The study provides technical detail about the LLM invocation, API configuration, request structure, or downstream processing.
- **IC6:** The study provides enough information to characterise at least one LLM code smell.
- **IC7:** The study has its full text available online.
- **IC8:** The study is peer reviewed for scientific literature.

Exclusion Criteria: We considered the following exclusion criteria:

- **EC1:** The study is a secondary source (e.g., literature review, survey, opinion piece, or conceptual commentary).

- **EC2:** The study does not describe any interaction with an LLM API, SDK, or integration mechanism.
- **EC3:** The study provides no implementation detail allowing us to identify or analyse integration defects.
- **EC4:** The study focuses exclusively on model training, dataset creation, benchmarking, or evaluation without addressing integration aspects.
- **EC5:** The study’s full text is not available online.
- **EC6:** The study does not provide any evidence of LLM integration behavior relevant to code smell extraction.
- **EC7:** The study is not written in English.

We applied our inclusion and exclusion criteria in two steps, first using the titles and abstracts, then the full texts. We acknowledge that some inclusion and exclusion criteria are logically complementary, for instance IC7 and EC5 describe opposing conditions. We explicitly state both inclusion and exclusion criteria separately to comply with the PRISMA guidelines, which recommend clearly articulating both positive and negative selection criteria to enhance transparency and reproducibility.

During the initial screening, we assessed the titles, abstracts, and page counts, and determined whether the papers qualified as primary. Two authors independently conducted the screening using predefined inclusion and exclusion criteria. To ensure consistency, we compared the screening results for a randomly selected subset of papers using Cohen’s Kappa. We obtained near-perfect agreement ($k=0.85$), demonstrating the reliability of the screening process. For the remainder of the screening, the authors met regularly to review results and resolved disagreements through discussion and consensus. After completing the initial screening, we selected the set of papers for full-text review as potential primary sources.

In the second round of screening, two authors independently examined papers in depth to verify whether it provided sufficient technical detail about LLM integration, including the invocation mechanism, the surrounding architectural or code-level context, and the observable integration behavior. To ensure a shared understanding of the criteria, we assessed the agreement between the two authors on 15 randomly selected papers using Cohen’s Kappa. We obtained a near-perfect agreement ($k=0.93$), demonstrating strong consistency in the interpretation and application of the criteria. This level of agreement allowed us to confidently proceed with the eligibility assessment for the remaining papers. Following this stage, we identified the set of primary papers that satisfied all eligibility requirements for inclusion in our evidence base.

4.1.4 Snowballing

We performed two iterations of backward and forward snowballing to identify additional relevant papers. This process led to the inclusion of 16 additional studies in the final selection. Overall, after completing all phases of PRISMA flow, we obtained a final selection of 27 papers.

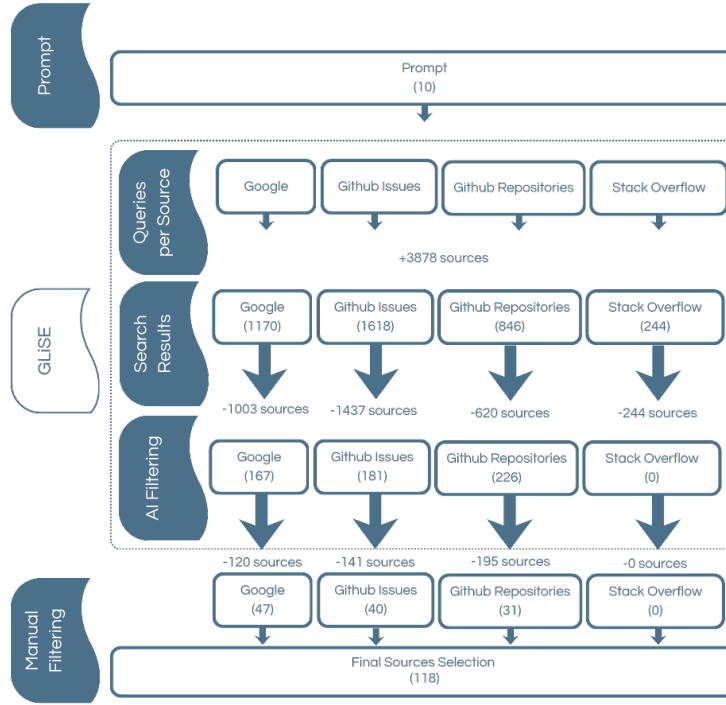


Fig. 3 Grey Literature Mining Flow.

4.2 Grey and Empirical literature Mining

Research literature provides a delayed and partial view of emerging integration practices. In contrast, grey and empirical literature such as engineering blogs, GitHub issues and repositories, programming forums, and official documentation captures real-world practices in near real time that are often overlooked in academic venues [26, 27]. Accounting for these sources is essential to software engineering research. Therefore, to capture real-world LLM integration practices in software systems, we complemented the systematic review with a structured mining of grey and empirical literature.

This allows us to explore new perspectives and to enrich the catalog by providing additional examples and counter-examples, as well as by documenting emerging integration issues not yet studied in academic venues, such as subtle misuses of provider features and configuration options. For each identified code smell, we associate relevant grey literature sources, as presented in the evidence table (Table 1).

We followed a strict methodology, comparable to the one applied for scientific papers. We relied on GLiSE [19], a three-step, prompt-driven and ML-powered tool for the extraction and filtering software engineering grey literature sources from Google, GitHub issues, GitHub repositories, and Stack Overflow posts. We used GLiSE

because, unlike general AI-based search tools such as Perplexity [28], it is specialized for software engineering grey literature and is fully traceable and reproducible, allowing the extraction of intermediate traces and artifacts at each step of the search process.

Our methodology is composed of three steps: (i) Prompts Creation, (ii) Extraction with GLiSE, and (iii) Manual Filtering. We first defined ten textual prompts, which were then used with GLiSE. Finally, we manually filtered the resulting corpus to refine the selection and ensure relevance. After this process, we completed the grey literature corpus with targeted manual searches using search engines and navigating the official documentation of major LLM and VLM providers, including OpenAI, Gemini, Anthropic, Hugging Face, and Ollama [29, 64, 122, 123, 220]. This allowed us to identify complementary evidence for potential code smells, assess their relevance, and consider providers’ guidance and practices. We focused primarily on official documentation, as these sources are both reliable and directly aligned with the subject of this study.

The complete grey literature mining workflow, along with the number of results and intermediate artefacts produced at each step, is shown in Figure 3. All artefacts and traces generated throughout this process are included in the replication package [30] to support reproducibility.

4.2.1 Prompts Creation

To extract relevant and useful grey and empirical literature, we created ten distinct textual prompts designed for use with GLiSE. Some prompts were intentionally generic, while others targeted specific features and functionalities of LLM and VLM APIs and SDKs. This allowed us to ensure broad coverage of LLM and VLM inference in software systems, while increasing the likelihood of identifying precise and relevant documentation related to suspected integration pitfalls. Using multiple prompts with varying levels of granularity mitigates the risk associated with relying on a single prompt that could be either too generic to find specific issues or too narrow to cover all relevant integration aspects. Therefore, by using multiple prompts, we aimed to maximize the relevance and quality of the retrieved sources.

All ten prompts are provided in the replication package [30].

4.2.2 Extraction with GLiSE

To search for grey and empirical literature, we used GLiSE [19] independently for each of the ten prompts created in the previous step, extracting and filtering sources from Google, GitHub, and Stack Overflow for each prompt. The final results obtained for each prompt were then grouped into a single corpus, while ensuring the absence of duplicates using a Python script.

GLiSE operates by first generating source-specific search queries from a textual prompt. These queries are then executed using the respective APIs of each source. Finally, retrieved results are screened based on their semantic relevance to the original prompt using the embeddings (vectorial representation of the semantics) of their metadata and machine learning classifiers. All generated queries, retrieved results, and

settings can be extracted to support reproducibility. We provide ours in the replication package [30].

After running GLiSE on the ten prompts and combining their final results, we obtained a total of 574 results, including 167 from Google Search, 181 from GitHub issues, 226 from GitHub repositories, and none from Stack Overflow.

4.2.3 Manual Filtering

After obtaining results using GLiSE, we performed an additional manual screening step because the documentation required for this study is highly specific, and our relevance criteria are stricter than general topical relevance. Our subject can also be confused with related but distinct concepts, such as code smells in code generated by LLMs, which can result in sources being erroneously filtered as relevant. We therefore combined GLiSE’s automated filtering, to speed up the process and enable broader searches, with manual screening to ensure the quality and reliability of the final grey literature corpus.

At the end of this step, 118 sources remained, including 47 from Google search, 40 from GitHub issues, 31 from GitHub repositories, and none from Stack Overflow.

5 LLM Code Smells

This section presents our catalog of LLM code smells. We first introduce the taxonomy that structures the catalog and clarifies the nature of the underlying smells. We then present the catalog of LLM code smells according to this taxonomy. Table 2 summarizes, for these nine code smells, their category in the taxonomy and their main effects on software quality. Extended examples, further discussion, and additional instances are available in our replication package [30]. New LLM code smells introduced in this paper are marked with *.

5.1 Taxonomy of LLM code smells

We organize the catalog into a taxonomy composed of three categories of LLM code smells, defined according to the nature of the underlying smell. More specifically, an LLM code smell may be primarily related to *Structural* or API usage, *Data semantics*, or *Protocol*. The categories were derived by grouping the identified smells according to the dimension of LLM integration they primarily concern, namely *Structural or API usage*, *Data semantics*, or *Protocol*. These categories also differ in how strongly they are coupled to provider-specific API designs, which may evolve over time. This taxonomy, illustrated in Figure 4, provides a conceptual structure for the catalog by clarifying why each smell arises during development and by indicating the kind of reasoning typically required for its detection.

Structural or API usage This category covers LLM code smells that arise from how developers invoke the LLM API, configure parameters, and compose requests. These source code defects are visible directly in the structure of the invocation, such as in parameter choices, the construction of the messages array, or the instantiation of helper objects. As a result, these smells can usually be identified through static

analysis of the call site, without requiring reconstruction of full data-flow or execution semantics.

As this category is directly tied to the composition of requests and the way the API is invoked, it is sensitive to API evolution. However, the aspects covered by the current smells in this category appear relatively established across the majority of providers, reflecting a certain degree of standardization that has started to take place.

Data semantics This category covers defects that arise from incorrect assumptions about the structure, validity, or stability of LLM inputs and outputs, including multi-modal data such as images or typed JSON payloads. These LLM code smells manifest when downstream components operate on non-validated or misinterpreted data, for example when free text is consumed as a structured JSON or when raw screenshots are sent without cropping or downscaling. Detecting these smells typically requires relating the LLM inference to how its inputs and output are parsed, stored, or executed.

The concepts underlying this category are relatively stable, as they mainly concern the structure and handling of input and output data of the LLM rather than provider-specific API conventions. These smells are therefore less subject to change as APIs and models evolve.

Protocol This category covers LLM code smells that concern the overall behavior of the LLM inference rather than a single invocation. These defects arise when defaults or unstable system-wide settings are used. Typical examples include implicit temperature, missing reasoning effort configuration, unbounded token budgets, or moving model aliases. Such defects affect reliability, reproducibility, and robustness across runs, as default values may evolve over time or differ between LLM providers.

The coupling to provider-specific behavioural parameters makes *Protocol* smells the most subject to change as APIs and models evolve among the three categories, as these parameters are more likely to change as the field matures. For some smells in this category, such variation across model types and versions is already observable.

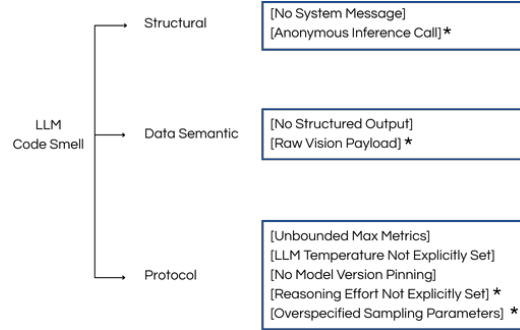


Fig. 4 Classification of LLM code smells

5.2 LLM Code Smells Catalog

This section presents the nine LLM code smells identified in our catalog. Each smell is described using a common structure composed of its context, problem, solution, example, taxonomy category, and evolutionary stability.

5.2.1 No Structured Output (NSO)

Context: LLM-Integrating systems often expect typed fields (e.g., JSON) but rely on LLM inference outputs which may not respect the format and produce raw free-form text. This smell applies when the output is later parsed, indexed, or executed assuming structure.

Problem: Without an enforced output schema, the system may receive free-form text where structured fields are expected [59]. This increases error-proneness via schema drift, missing or renamed fields, type mismatches, and silent truncation that passes as success, breaking parsers and downstream steps. It degrades reliability as runs become inconsistent, data stores accumulate corrupted or hallucinated values, and execution/storage/display paths face injection [61].

Solution: Enforce structured output at the API boundary. With OpenAI, declare a JSON Schema via `response_format` (chat completions) or `text_format` (responses). With the Python SDK, you may instead bind the format directly to classes [66]. Always validate results to handle refusals or other errors [60, 62, 66].

Example: In Listing 1, the faulty version consumes free-form text, whereas the corrected version enables strict parsing and safer downstream use by enforcing a JSON schema via `response_format`.

```
1 # Define a JSON schema e.g. result_schema = ...
2 response = openai.chat.completions.create(
3     model="gpt-4o-2024-11-20", messages=messages)
4 response = openai.chat.completions.create(
5     model="gpt-4o-2024-11-20", response_format={
6         "type": "json_schema",
7         "json_schema": {"name": "Result", "schema":
8             result_schema}}, messages=messages)
```

Listing 1 No Structured Output (NSO)

Taxonomy Category: *Data semantics*. NSO belongs to this category because its core defect is semantic rather than purely structural. The system assumes structured data, such as JSON objects or typed fields, while the LLM call only guarantees free-form natural language output. The smell therefore arises from a mismatch between the semantic expectations of downstream components and the actual output contract enforced at the inference boundary.

Evolutionary Stability: The core idea behind NSO, which is enforcing data structure and its validation before parsing, should remain consistent as the field of LLMs evolves. Additionally, structured output functionalities are becoming increasingly common in LLM APIs and SDKs.

5.2.2 Unbounded Max Metrics (UMM)

Context: Hosted LLM APIs (e.g., OpenAI, Anthropic) expose finite token windows, per-request output caps, and rate limits. Pipelines that ignore these constraints, or omit their own limits on concurrency and response size, are prone to throttling and partial outputs.

Problem: Leaving token budgets, timeouts, and retries unbounded undermines system robustness and performance. Unspecified token budgets may yield outputs that exceed context limits (truncating mid-structure or return 400), overload downstream parsers, or inflate token costs and memory usage [121]. Not specifying timeouts and retry limits can cause requests to hang indefinitely, leading to unpredictable latency, rising costs, and reduced throughput as resources are tied up by long-running calls [125]. Not specifying these values also hinders maintainability, as defaults may change over time and tracking settings is essential for reproducibility.

Solution: Always bound and adjust the `max_output_tokens`, `timeout`, and `max_retries` parameters [126]. Monitoring the number of input tokens is also recommended.

Example: In Listing 2, the faulty version leaves metrics unbounded, whereas the corrected enforces bounded metrics.

```
1 client = OpenAI()
2 response = client.responses.create(model=
3     "gpt-4o-2024-11-20", input=prompt)
4 client = OpenAI(timeout=20, max_retries=3)
5 response = client.responses.create(
6     model="gpt-4o-2024-11-20",
7     input=prompt, max_output_tokens=256)
```

Listing 2 Unbounded Max Metrics (UMM)

Taxonomy Category: *Protocol*. UMM belongs to this category because its core issue concerns the operational contract that governs LLM inference rather than the local structure of a single call. The smell arises when essential execution bounds, such as `max_output_tokens`, `timeout`, and `max_retries` parameters, are left implicit or unspecified. It therefore affects the stability, robustness, and predictability of inference behaviour across executions, which makes it a protocol-level smell rather than a purely structural one.

Evolutionary Stability: The core idea behind UMM, that the resources given to the inference should be specified to avoid unwanted repercussions, is not new to LLMs and should remain consistent as the field changes. The details of how this is applied may change and improve, but the principle of accordingly limiting request retries and token budget should remain.

5.2.3 LLM Temperature Not Explicitly Set (TNES)

Context: In LLM APIs, SDKs, and runtimes such as OpenAI, Hugging Face, or Ollama, temperature controls sampling stochasticity [144] and is a key decoding parameter, together with nucleus sampling (`top_p`) and top k sampling (`top_k`).

Problem: Relying on an implicit temperature reduces maintainability and reliability. Defaults temperature differ across providers/models [64, 123, 146] and may

change over time [143], which harms reproducibility, portability and can silently alter behavior.

Solution: Always specify explicitly the temperature while minimizing node coupling. If temperature is used, always specify it explicitly and justify its value: low (0–0.3) for precise, repeatable automation, higher (0.7–1.0) for creative generation, and avoid extremes [142]. If `top_p` or `top_k` is explicitly set, do not additionally set temperature, and instead document that randomness is governed by `top_p` or `top_k` to avoid overspecification and erratic behaviors [164].

Example: In Listing 3, the faulty version omits temperature, while the corrected version makes it explicit.

```
1 response = openai.chat.completions.create(  
2     model = "gpt-4o-2024-11-20", messages = messages )  
3     model = "gpt-4o-2024-11-20", messages = messages, temperature = 1.0 )
```

Listing 3 LLM Temperature Not Explicitly Set (TNES)

Taxonomy Category: *Protocol*. TNES belongs to this category because its core defect concerns the behavioural configuration of LLM inference rather than the local structure of the invocation. Temperature is a primary sampling parameter that directly affects reproducibility, output stability, and behavioural consistency. When it is left implicit, the system silently relies on provider- or model-specific defaults, which may vary across APIs, environments, or model updates. TNES is therefore a protocol-level defect because it weakens the explicit inference contract that governs system behaviour over time.

Evolutionary Stability: The temperature parameter has been a relatively established standard across LLM APIs for some time. However, its use has recently been removed for some models, such as certain OpenAI reasoning models [64]. This suggests that this smell may become obsolete for some specific model families as the field evolves.

5.2.4 No Model Version Pinning (NMVP)

Context: In LLM APIs, runtimes, and hubs (e.g. OpenAI, Ollama, Hugging Face), models can be called via moving aliases (e.g., *gpt-4o*) or immutable versions/snapshots (e.g., *gpt-4o-2024-11-20*). Aliases may advance as providers update models [173].

Problem: Using only a provider alias removes explicit versioning, so weights, prompts, and safety filters can change without notice and shift behavior [172]. It reduces maintainability by eroding traceability and reproducibility. Runs cannot be tied to a stable model build and portability across environments degrades as the same alias may yields different behavior.

Solution: Always specify an immutable identifier and record it with other run metadata. Update versions via change control [64, 145, 169].

Example: In Listing 4, the faulty version uses a moving alias, whereas the corrected version pins an immutable version/snapshot to ensure reproducibility and traceability.

```
1 response = openai.chat.completions.create(  
2     model="gpt-4o", messages=messages )  
3     model="gpt-4o-2024-11-20", messages=messages )
```

Listing 4 No Model Version Pinning (NMVP)

Taxonomy Category: *Protocol*. NMVP belongs to this category because its core defect concerns the stability of the inference contract over time rather than the immediate structure of a single invocation. When a system relies on a moving model alias instead of an immutable version or snapshot, its effective behaviour may change without any modification to the source code. NMVP therefore weakens reproducibility, traceability, and behavioural stability, which makes it a protocol-level defect.

Evolutionary Stability: The principle behind NMVP is well rooted in established SE practices, where explicit versioning has long been a fundamental requirement for reproducibility and traceability. The aliasing versus pinning distinction remains consistent across major LLM providers, making this smell among the least subject to change as the field evolves.

5.2.5 No System Message (NSM)

Context: In role-based chat APIs and runtimes, the system message sets global behavior, constraints, and tone for the assistant.

Problem: Without a system message, the model lacks high-level guidance, which reduces consistency and adherence to constraints. Outputs become more generic and harder to control. This degrades reliability, as additional iterations or longer prompts are often required to achieve adequate results [179].

Solution: Always include a clear system message that defines roles, goals, and constraints. Keep task specifics in the user message [177].

Example: In Listing 5, the faulty version omits a *system* message, whereas the corrected version adds a concise system instruction to anchor behavior, improve consistency and response quality.

```
1 response = openai.chat.completions.create(  
2     model="gpt-4o-2024-11-20",  
3     messages=[{"role": "user", "content": "Explain recursion with an example"}])  
4     messages=[{"role": "system", "content": "You are a Computer Science tutor. Answer  
5     "clearly."}, {"role": "user", "content": "Explain recursion with an example"}])
```

Listing 5 No System Message (NSM)

Taxonomy Category: *Structural or API usage*. NSM belongs to this category because its core defect lies in how the LLM request is structurally constructed at the API level. In role-based chat interfaces, the system message is an explicit element of the message array that defines global behaviour, constraints, and tone. Omitting it is therefore a defect in request composition rather than a semantic mismatch in the exchanged data or a protocol-level inference policy issue.

Evolutionary Stability: The concept of system messages has been more and more standardised across models and providers. Even if the concrete implementation at the API level may vary slightly between APIs or SDKs, the core concept remains the same. This seems to indicate that this practice is there to stay and shouldn't change much as the field of LLMs evolve.

5.2.6 Reasoning Effort Not Explicitly Set (RENES) *

Context: Modern RLMs providers expose controls on the depth or intensity of internal reasoning. Depending on the provider, these controls can be expressed as a parameter such as *reasoning effort* or *reasoning depth*, as a budget on internal reasoning tokens or steps, or as a dedicated reasoning or thinking mode that can be switched on or off. These influence how much structured internal reasoning the model performs before producing user visible tokens, which directly affects accuracy, robustness for complex tasks, latency, and cost [190]. In many APIs, the default value of these controls is model specific and may change over time as models and deployments evolve [190, 191].

Problem: When a reasoning capable model is invoked without explicitly configuring its reasoning control, the system silently relies on provider specific defaults. Since these defaults differ across models and can be updated without any change in the client code, migrating to a new model or endpoint can alter the effective depth of reasoning, as well as runtime and cost, in non transparent ways. This harms reproducibility because executions cannot be tied to a clearly specified reasoning configuration. It also harms maintainability because small changes in the model name or deployment can change behavior without a visible code change. Finally, it harms reliability, since developers and stakeholders may believe that the system systematically uses deep reasoning while, in practice, it may run with a shallow or even disabled reasoning mode [192].

Solution: Use the reasoning control as part of the public contract of any call to a reasoning capable model. Always set the corresponding parameter explicitly in code and, when relevant, document the chosen level in configuration files and experimental protocols. For interactive or high throughput scenarios, select a lower reasoning effort or shallower reasoning depth to keep latency and cost under control and validate the choice empirically. For complex problem solving, high risk, or safety critical tasks, explicitly request higher reasoning effort. When providers introduce new reasoning levels or change their defaults, update code and documentation through explicit change management instead of relying on new defaults. Conceptually, the reasoning control should be handled as the primary parameter for depth of reasoning for these models, in the same way that temperature is treated as a primary parameter for sampling behavior in classical LLMs [119, 193].

Example: In the following example Listing 6 , the faulty version calls a reasoning model without specifying any reasoning configuration, while the corrected version pins *reasoning_effort* to a value that reflects an explicit trade off between latency, cost and accuracy. The example is adapted from the GPT 5 cookbook from OpenAI.

```
1 client = OpenAI()
2 response = client.responses.create(
3     model="gpt-5.1-mini-2025-02-01",
4     input=[
5         {
6             "role": "system",
7             "content": "You are a careful assistant that explains your reasoning
8             ↪ for complex software engineering decisions."
9         },
10        {
11            "role": "user",
12            "content": (
```

```

12         "We plan to integrate a large language model into a safety
13         ↳ critical pipeline for code review. Explain the main tradeoffs, risks, and
14         ↳ mitigations in a structured way."
15     },
16     ],
17     temperature=0.2 )
18     temperature=0.2,
19     reasoning={"effort": "minimal"},
20 )

```

Listing 6 Reasoning Effort Not Explicitly Set (RENES)

Taxonomy Category: *Protocol*. RENES belongs to this category because its core defect concerns the explicit control of reasoning behaviour during inference rather than the local structure of the invocation. In reasoning models, parameters such as *reasoning_effort*, *reasoning_depth*, or *thinking_mode* govern the trade-off between accuracy, latency, and cost. When these controls are left implicit, the system silently depends on provider-specific defaults that may vary across models, endpoints, or updates. RENES is therefore a protocol-level defect because it weakens the explicit inference contract that should govern reasoning behaviour over time.

Evolutionary Stability: The way reasoning effort is constrained in reasoning models currently lacks standardization across providers and models. Depending on the provider, the core mechanism used differs or may not even exist [31, 32, 123]. This suggests that further changes are likely to come that may impact the concrete manifestation of this smell. However, the core idea of explicitly controlling the amount of reasoning allocated to a task should remain relevant as the field evolves.

5.2.7 Raw Vision Payload (RVP) *

Context: Vision-enabled LLM APIs allow sending images such as UI screenshots, diagrams, or camera photos together with text. It is straightforward for developers to pass any bytes or encoded image directly as input. In practice, many systems send full resolution screenshots or raw images captured from large displays or high resolution devices, even when only a small region is relevant. Since vision inputs are internally converted to tokens, image size, resolution, detail level, and the number of images have a strong impact on latency, cost, and how much of the multimodal context window is consumed [194–196].

Problem: When systems systematically send raw, unbounded vision payloads, the effective behavior of the application depends on how the provider handles very large images or many images in a single request. This increases latency and token usage, which can create cost spikes and rate limit issues. It also harms reliability, since the model may fail to focus on a small but critical detail in a large screenshot, and truncation of multimodal context may occur in ways that are opaque to the developer. Finally, unfiltered screenshots can leak unnecessary or sensitive interface elements, which complicates compliance and privacy reviews [194, 195].

Solution: Treat vision input as a limited resource and introduce explicit budgets for image size and content. Crop images to the region that is relevant for the task, downscale large screenshots to a reasonable maximum resolution, and avoid sending multiple redundant images in the same call. When the API exposes configuration for

image detail, explicitly select an appropriate level rather than relying on defaults, for example using a low detail setting for coarse inspection and reserving a high detail setting for cases that require fine grained visual analysis. For tasks that primarily depend on text, prefer extracting text or structured data from the source instead of sending a screenshot of that text. Implement preprocessing in a dedicated helper rather than inlining raw image bytes in the call, and document the chosen image budgets alongside text context and output limits so that multimodal behavior is reproducible and auditable [196].

Example: Listing 7 illustrates Raw Vision Payload (RVP), where the faulty version sends full resolution screenshots directly to the vision model while the corrected version first crops and resizes the image and configures a low detail setting to respect an explicit input budget.

```

1  from openai import OpenAI
2
3  client = OpenAI()
4
5  def describe_bug(screenshot_bytes):
6      small_image = resize_and_crop(screenshot_bytes, max_side=1024)
7      response = client.responses.create(
8          model="gpt-4.1-mini",
9          input=[
10             {
11                 "role": "user",
12                 "content": [
13                     { "type": "input_text",
14                       "text": "Describe the bug shown in this screenshot."
15                     },
16                     { "type": "input_image",
17                       "image": screenshot_bytes,
18                     },
19                     { "type": "input_image",
20                       "image": small_image,
21                       "detail": "low",
22                     },
23                 ],
24             },
25             ],
26             temperature=0.2,
27         )
28         return response.output[0].content[0].text
29 
```

Listing 7 Raw Vision Payload (RVP)

Taxonomy Category: *Data semantics.* RVP belongs to this category because its core defect concerns the semantic suitability of multimodal inputs rather than the structural form of the API call. Raw screenshots or full-resolution images are passed to the model as if their content, granularity, and size were already appropriate for the intended task. The smell therefore arises from a mismatch between the visual payload that is actually provided and the semantic assumptions made by the system about what the model should attend to and process reliably.

Evolutionary Stability: The core principle behind RVP does not depend on any single API parameter, making the concept of this smell relatively stable. However, the specific mechanisms available to control image detail and resolution vary between

providers and may evolve over time, meaning the concrete instances of this smell could change as the field evolves [33, 34, 86].

5.2.8 Overspecified Sampling Parameters (OSP) *

Context: Modern LLM runtimes expose multiple sampling controls that shape the stochasticity of decoding, most commonly temperature, nucleus sampling (`top_p`), and top-k sampling (`top_k`). These parameters are not independent, since they all restrict or reweight the next token distribution, and their exact semantics and supported combinations vary across providers and model families [35, 197–199].

Problem: OSP occurs when a call pins several randomness node at once, for example setting temperature together with `top_p` or `top_k`. This overspecification makes the effective decoding policy hard to reason about and difficult to reproduce across providers, because small API changes, different default behaviors, or different implementations of the same parameter can alter which constraint dominates. In practice, OSP reduces maintainability and portability, complicates debugging and tuning, and can introduce silent quality regressions where output diversity or determinism shifts without a clear causal explanation [157, 203].

Solution: Prefer a minimal, explicit sampling contract that uses a single primary control for randomness and documents the rationale. If temperature is the chosen control, set it explicitly and tune by task (e.g. Listing 3) for creative generation, while avoiding extremes [142]. If `top_p` or `top_k` is used, keep temperature unset and document that randomness is governed by `top_p` or `top_k`. Centralize these choices and validate changes with small regression suites to ensure that decoding behavior remains stable across model and provider updates [162, 204].

Example: Listing 8 illustrates Overspecified Sampling Parameters (OSP), where the faulty version explicitly sets multiple sampling controls (e.g., `temperature` and `top_p` or `top_k`) even though they are alternative mechanisms for shaping randomness. The corrected version keeps a single primary control (`temperature`) to avoid unintended interactions and to make the generation behaviour easier to reason about and reproduce.

```
1  import anthropic
2
3  client = anthropic.Anthropic()
4
5  def generate_creative_story(topic):
6      message = client.messages.create(
7          model="claude-3-5-sonnet-20241022",
8          max_tokens=1024,
9          messages=[
10             {"role": "user", "content": f"Write a sci-fi story about {topic}"}
11         ],
12         temperature=0.9,
13         top_p=0.95, #or top_k
14         temperature=0.9, #or top_p=0.95,
15         return message.content[0].text
```

Listing 8 Overspecified Sampling Parameters (OSP)

Taxonomy Category: *Protocol*. OSP belongs to this category because its core defect concerns the sampling policy that governs inference behaviour rather than the local

structure of a single invocation. Parameters such as `temperature`, `top_p`, and `top_k` act as competing controls over decoding stochasticity, and combining several of them without a clear rationale weakens the clarity and stability of the inference contract. OSP is therefore a protocol-level defect because it makes model behaviour harder to reason about, reproduce, and tune consistently across providers and updates.

Evolutionary Stability: While the underlying principle of OSP, keeping the sampling policy explicit and minimal, should remain relevant as the field evolves, its concrete application may change over time. As the interactions between parameters such as `temperature` and `top_p` become better understood, what constitutes overspecification may be redefined. Additionally, as noted for TNES, `temperature` may not be available for all model families. OSP is therefore among the smells more subject to change as the field matures.

5.2.9 Anonymous Inference Call (AIC) *

Context: Many LLM providers expose optional request fields that allow applications to attach a stable end-user identifier or equivalent metadata to each inference call, enabling traceability, abuse monitoring, and accountable auditing across sessions and tenants. In practice, such identifiers are also useful for correlating logs, reproducing incidents, and performing user scoped analytics in multi user systems [213–215].

Problem: AIC arises when an application performs inference without attaching any user level identifier, even though the surrounding code clearly operates in a multi user context (e.g., a web request, session id, or authenticated principal). This omission reduces maintainability and reliability because failures, hallucination reports, cost spikes, and rate limit events cannot be attributed to a specific user or session, which weakens incident response and hampers debugging and evaluation. From a governance perspective, anonymous calls also reduce accountability and complicate safety workflows such as detecting misuse patterns and enforcing user specific throttling [205–207].

Solution: Propagate an explicit user identifier from the application boundary to every LLM inference and document the traceability contract. Concretely, when a request is associated with a session or authenticated user, include a stable identifier in the inference call, as illustrated in Listing 9. In practice, this identifier is typically a pseudonymous token such as a hash of the user email (or an internal account id), rather than raw personal data. Keep the mapping in application storage, and ensure consistency across providers by routing the identifier through a single client wrapper. Finally, log the identifier alongside model name, sampling parameters, and request ids so that debugging, auditing, and abuse monitoring remain feasible under provider changes and evolving deployment configurations [208–212].

Example: Listing 9 illustrates Anonymous Inference Call (AIC), where the faulty version issues an LLM request without attaching any user level identifier, which weakens traceability and makes it harder to audit usage, investigate incidents, or enforce per user policies. The corrected version provides an explicit `user` identifier to associate the request with the originating session and enable accountability and monitoring.

```
1 from openai import OpenAI
2
```

```

3  client = OpenAI()
4
5  def get_response(user_session_id, prompt):
6      response = client.chat.completions.create(
7          model="gpt-4o",
8          messages=[{"role": "user", "content": prompt}],
9          temperature=0.7,
10         user=user_session_id # Traceability: Link execution to the specific user
11     )
12     return response.choices[0].message.content

```

Listing 9 Anonymous Inference Call (AIC)

Taxonomy Category: *Structural or API usage*. AIC belongs to this category because its core defect lies in how the inference request is constructed at the API level. Many providers expose explicit attribution fields such as `user` or `user_id` to associate a request with an application-level principal or session. Omitting this information, despite its availability in the surrounding code, is therefore a defect in request composition rather than a semantic mismatch in the exchanged data or a protocol-level inference policy issue.

Evolutionary Stability: While the principle of tracking users to ensure proper auditing of usage and abuse is there to stay, the way it is concretely implemented across APIs is still actively changing. For example, the `user` parameter as described in this smell is being deprecated in favor of `safety_identifier` and `prompt_cache_key` in OpenAI’s Chat Completions API [31]. This smell is therefore currently at high risk of obsolescence, even if the concrete mechanisms may eventually stabilize and standardize across providers.

The nine instrumented LLM code smells have been introduced above with their context, problem, solution, example, and taxonomy category. To complement these individual descriptions, Table 2 summarizes their main software quality effects. More specifically, it shows whether each smell primarily impacts robustness, performance, maintainability, and reliability, thereby providing a compact cross-smell view of the catalog.

6 Detection Approach with SpecDetect4LLM

This section explains how we built our LLM code smell detection tool, *SpecDetect4LLM*, based on an existing tool for AI-Specific code smells, *SpecDetect4AI* [11]. In this section, we briefly explain the underlying DSL, describe how we instantiated rules for the smells in the taxonomy, and discuss the limits of the current tool, with a focus on intra file occurrences that can be captured by static analysis.

6.1 From the *SpecDetect4AI* DSL to *SpecDetect4LLM*

SpecDetect4AI models each code smell as a rule over the abstract syntax tree (AST) of a single Python file across the system. Rules are written in a small domain specific language (DSL) where variables range over AST nodes and where primitive predicates express syntactic or local control flow properties. The compiler translates a specification into a Python visitor that traverses one AST, enriches it with parent links and line numbers, and reports a violation whenever the rule body holds for some node.

Table 2 Overview of the taxonomy and effects of LLM code smells.

Code Smell	Taxonomy	Effect			
		RO	P	M	R
No Structured Output (NSO)	Data semantics	✓			✓
Unbounded Max Metrics (UMM)	Protocol	✓	✓	✓	
LLM Temperature Not Explicitly Set (TNES)	Protocol			✓	✓
No Model Version Pinning (NMVP)	Protocol			✓	✓
No System Message (NSM)	Structural or API usage			✓	✓
Reasoning Effort Not Explicitly Set (RENES) *	Protocol	✓	✓	✓	✓
Raw Vision Payload (RVP) *	Data semantics		✓	✓	
Overspecified Sampling Parameters (OSP) *	Protocol			✓	✓
Anonymous Inference Call (AIC) *	Structural or API usage	✓		✓	

* *RO*: Robustness operational resilience, error-prone behavior

* *P*: Performance cost, latency, memory

* *M*: Maintainability reproducibility, portability, observability

* *R*: Reliability response quality, consistency, stability over time

SpecDetect4LLM keeps this overall architecture intact. The main design effort consists in adding a library of LLM-oriented predicates so that each smell in the taxonomy can be expressed as one rule in the DSL. We adopted a pragmatic strategy. Whenever the existing predicates from *SpecDetect4AI* were sufficient, we reused them unchanged. New predicates were added only when needed to encode a concrete LLM smell. This keeps the rule language compact and facilitates maintenance.

The new predicates fall into three intuitive groups. First, classification predicates identify LLM related calls and distinguish specialised families. For example, `isLLMCall` and `isTextGeneratingCall` recognise provider-specific text generation calls through their call path and simple alias analysis inside the file. Predicates such as `isVisionModelCall` and `isReasoningModelCall` refine this classification by checking model identifiers and constructor patterns for vision capable and reasoning capable models.

Second, configuration predicates test whether specific parameters are set at the call site. `hasNoTemperatureParameter` checks for missing temperature, `hasNoModelVersionPinning` checks that model identifiers are not version pinned, `hasNoReasoningEffort` detects calls to reasoning models without effort or thinking depth, `hasNoBoundedMetrics` focuses on resource limits such as output length or timeout, and `hasNoStructuredOutput` indicates that no structured response format or parser is attached. These predicates inspect both direct keyword arguments and simple dictionary based configuration, using a backward scan to recover the latest assignment to a configuration dictionary in the same file.

Third, prompt and payload predicates characterise chat like prompts and multimodal content. `isRoleBasedLLMChat` identifies role based chat invocations, while `hasNoSystemMessage` checks whether any system message or instruction can be recovered from the call or from a local configuration dictionary. For multimodal prompts, `hasImageContent` detects the presence of image payloads, `hasImagePreprocessing`

searches for evidence of preprocessing pipelines, and `hasExplicitDetailLevel` checks whether a detail or quality level is configured. All these predicates operate within the AST and rely on simple local reasoning such as last assignment and pattern based recognition of image processing calls.

6.2 Rule specification and alignment with the taxonomy

Each smell in the taxonomy is encoded as a single declarative rule that combines a small set of predicates. As in *SpecDetect4AI*, rules are written as quantified formulas over the AST. Variables range over calls or small local control-flow fragments inside the file, and the rule body is expressed as a conjunction of predicates that makes the symptoms of the smell explicit. Our implementation extends the predicate library to capture the diversity of LLM SDK surfaces and orchestration patterns observed in repositories. This design keeps the rules local, auditable, and directly aligned with the taxonomy introduced above.

No Structured Output.

The NSO rule detects generation calls whose outputs are likely to be consumed as structured artifacts without an explicit output contract at the inference boundary. It relies on LLM-call classification predicates to identify text-generation calls and then applies `hasNoStructuredOutput` to determine whether the call lacks a structured response format, output parser, JSON schema, or equivalent local validation mechanism. The predicate inspects direct keyword arguments, locally assigned configuration dictionaries, and common SDK-specific fields used to request structured outputs. NSO is aligned with the Data semantics category because the rule captures a mismatch between the expected structure of downstream data and the absence of an explicit structure enforced at the LLM call.

Unbounded Max Metrics.

The UMM rule detects LLM calls for which no local resource or execution bound can be recovered. It first identifies LLM inference calls using `isLLMCall`, then reports a violation when `hasNoBoundedMetrics` holds. This predicate checks whether the call or its locally recoverable configuration specifies limits such as output length, timeout, retry budget, or provider-specific generation bounds. It also accounts for nested configuration styles, including Gemini generation dictionaries and wrapper-level options when they are visible in the same file. UMM is aligned with the Protocol category because it concerns the explicit operational contract that governs latency, cost, and robustness during inference.

LLM Temperature Not Explicitly Set.

The TNES rule targets text-generation calls for which temperature is required but not explicitly configured. It restricts attention to generative endpoints using `isTextGeneratingCall`, which recognises OpenAI-style `chat.completions.create` and `responses.create`, Gemini `generate_content`, common orchestration entry-points such as `invoke` and `predict`, and variable calls originating from a Hugging Face `pipeline("text-generation")`. The rule then reports a violation when

`isLLMCallRequiringTemperature` holds and `hasNoTemperatureParameter` is true. Constructor-like calls and pipeline constructors are explicitly guarded against to avoid spurious alerts. TNES is aligned with the Protocol category because temperature defines part of the sampling contract and affects reproducibility, stability, and behavioral consistency across executions.

No Model Version Pinning.

The NMVP rule detects LLM calls that rely on moving model aliases instead of stable model versions or snapshots. The rule first restricts the search space to calls where model versioning is meaningful, then applies `hasNoModelVersionPinning`. This predicate checks direct model arguments, locally assigned model variables, and simple configuration dictionaries when the model identifier can be recovered from the same file. The rule reports a violation when the model identifier corresponds to an alias or non-versioned family name rather than an immutable version. NMVP is aligned with the Protocol category because model identifiers define part of the inference contract and directly affect traceability, reproducibility, and behavioral stability over time.

No System Message.

The NSM rule detects role-based chat calls that omit a system-level instruction. It combines `isRoleBasedLLMChat`, which recognises chat-style message arrays, with `hasNoSystemMessage`, which searches the local call context for a `system` role or an equivalent provider-specific instruction field. The rule reports a violation when the request is composed only from user or assistant messages and no recoverable global instruction is present. NSM is aligned with the Structural or API usage category because the relevant evidence lies in the request structure itself, especially in the construction of the message array passed to the LLM call.

Reasoning Effort Not Explicitly Set.

The RENES rule detects reasoning-capable model calls that do not explicitly configure the amount or depth of reasoning. It first applies `isReasoningModelCall`, which recognises multiple provider-specific naming conventions, including OpenAI reasoning-oriented model families and pinned GPT-5 variants, Anthropic thinking-capable models, and Gemini thinking variants. The rule then applies `hasNoReasoningEffort`, which searches for direct reasoning arguments and nested configuration objects such as `generation_config`. RENES is aligned with the Protocol category because reasoning effort controls the trade-off between accuracy, latency, and cost, and leaving it implicit weakens the inference contract of reasoning-capable systems.

Raw Vision Payload.

The RVP rule detects vision-capable LLM calls that receive image payloads without visible preprocessing or explicit detail control. It combines `isVisionModelCall` with `hasImageContent`, then reports a violation when neither `hasImagePreprocessing` nor `hasExplicitDetailLevel` holds. The `isVisionModelCall` predicate recognises vision-capable providers and wrappers, while `hasImageContent` performs structural

search over nested dictionaries, lists, common image encodings, and URL-based payload patterns. RVP is aligned with the Data semantics category because it concerns the semantic suitability of multimodal inputs before they are sent to the model, including whether the image content is bounded, selected, and appropriate for the intended task.

Overspecified Sampling Parameters.

The OSP rule detects generation calls that configure several randomness controls simultaneously. It is implemented through `hasOverspecifiedSampling`, which identifies calls that set `temperature` together with `top_p` or `top_k`, either directly on the call or through an enclosing configuration context such as `with_options`. The rule focuses on cases where the local decoding policy becomes harder to interpret because multiple sampling mechanisms are pinned at the same time. OSP is aligned with the Protocol category because it concerns the sampling policy that governs inference behavior and affects reproducibility, portability, and tuning consistency across providers.

Anonymous Inference Call.

The AIC rule detects LLM calls executed in an apparent multi-user context without local user attribution. It combines `hasMultiUserContext`, which approximates the presence of request-scoped or session-scoped identifiers by searching enclosing scopes for patterns such as `request.user.id` and `session["user_id"]`, with `hasUserAttribution`, which checks whether the call propagates a user identifier through explicit `user` parameters, metadata fields, safety identifiers, or keys passed through `**kwargs`. The rule reports a violation when an LLM call occurs in a multi-user context while lacking any recoverable attribution field. AIC is aligned with the Structural or API usage category because the defect concerns how the inference request is constructed at the API level, especially whether available attribution information is propagated as part of the call.

Overall, this rule specification follows the same structure for all nine LLM code smells. Each rule identifies a relevant LLM inference context, checks for the absence or problematic combination of local evidence, and reports a violation only when the smell can be justified from the analysed file. This design preserves the lightweight nature of *SpecDetect4LLM* while maintaining a direct correspondence between the implementation and the taxonomy. Structural or API usage smells are detected from the shape of the request and its immediate arguments. Data semantics smells are detected by relating LLM calls to the structure and constraints of their inputs or outputs. Protocol smells are detected by checking whether configuration choices that govern behavior across executions are made explicit at the call boundary.

6.3 Scope of *SpecDetect4LLM*

As in *SpecDetect4AI*, the current *SpecDetect4LLM* tool is intentionally scoped as a lightweight AST-based static analyser. Each rule is evaluated on the AST of a single file and relies only on locally recoverable evidence, such as call structure, argument values, local control constructs, node relations, line numbers, and limited last-assignment recovery. This observation model is well suited to LLM code smell occurrences that

are locally explicit at the call site or in its immediate context. This design choice preserves scalability, keeps reported smells easy to localise and inspect, and supports large-scale analysis over open-source systems. It also makes detection explainable, since each reported smell can be traced back to source-level evidence visible in the analysed file. Accordingly, the current prototype targets statically observable local occurrences rather than full system-level behaviour. occurrences that depend on information distributed across files, propagated through interprocedural control-flow or data-flow relations, or materialised through external configuration artefacts remain outside its scope. Despite this restricted observation model, the tool already achieves encouraging precision and recall on the smells targeted in this study. Richer forms of static reasoning could further improve coverage for such non-local occurrences, but incorporating them would have substantially enlarged the implementation and evaluation scope of the present work and was therefore left outside the scope of this paper.

7 Validation Design and Results

In this section, we present the study that aims to validate our tool-based approach. We follow a mixed-method methodology through quantitative and qualitative data collection and analysis.

7.1 Research Questions

We aim to respond to the following research questions:

- ***RQ₁*: How precise is *SpecDetect4LLM* to detect LLM code smells?** This question assesses whether *SpecDetect4LLM* can accurately detect LLM code smells.
- ***RQ₂*: Does *SpecDetect4LLM* scale efficiently across diverse LLM-integrating systems, regardless of system size?** This question measures the runtime performance and scalability of the tool across a large set of LLM-integrating systems.
- ***RQ₃*: How prevalent are LLM code smells in open-source software systems that integrate LLMs, and which LLM code smells occur most frequently?** This question quantifies the relative prevalence of each smell in our corpus.

7.2 Objects of the Validation

We apply *SpecDetect4LLM* to a large-scale corpus of LLM-integrating systems through a two-source construction that combines targeted GitHub mining with reuse of a previously curated corpus. For the GitHub mining phase, we implement an automated collector that queries the GitHub Search API and extracts a target sample of 500 repositories. Candidates are retrieved using a set of LLM and multimodal keywords and topics, including *openai*, *anthropic*, *gemini*, *ollama*, *vllm*, *llava*, *vlm*, *lvm*, and phrases such as *vision language* and *reasoning model*, expressed as multiple search queries over repository metadata and README content. To avoid low-signal projects

and reduce noise, we filter out forks and archived repositories and enforce a minimum popularity threshold `stars:≥3`. We further prioritize recently maintained repositories by retaining projects with at least one push in the last 365 days, and by sorting candidates by the most recent push timestamp in descending order. To mitigate false positives due to keyword-only matches, we verify likely integration by fetching the repository README and probing common dependency manifests and lockfiles when available.

To strengthen external validity and mitigate sampling bias induced by GitHub search and ranking heuristics, we complement our crawl with 100 repositories from Shao et al. [2]. Their corpus captures widely used and previously studied systems, while our crawl emphasizes more recent and actively maintained projects, thereby increasing diversity in both project maturity and development activity. We then merge the two sources and remove duplicates (8 repositories). Duplicates include identical `owner/repo` identifiers and near-duplicates introduced by forks or mirrors of the same codebase. Next, we harmonize repository metadata across both subsets. Concretely, we standardize fields such as stars, forks, last update timestamps, the default branch, and the reference commit used for analysis. After fork removal, duplicate filtering, and metadata harmonization, the final dataset contains 692 repositories spanning both established and emerging LLM-integrating systems and supporting consistent downstream validation.

7.3 Sampling Design and Ground-Truth Construction

As shown in Section 7.2, our full evaluation corpus contains 692 systems, 171,194 source files, and 40,919,625 LOC, targeting the functional Python components of the repository. Producing a complete line-level ground truth for the entire corpus would require well over 5,000 human-hours when extrapolating from the annotation throughput reported in [36], which is infeasible within a single study.

To keep the workload tractable while preserving statistical validity, we construct a stratified random sample across repositories. This choice explicitly mitigates sampling bias. A global simple random draw would be dominated by large repositories and could systematically under-represent smaller ones. In contrast, stratified sampling ensures that each repository contributes to the sample proportionally to its size, improving representativeness across heterogeneous projects.

We set the target sample size using Cochran’s formula for proportions [37]. With a 95% confidence level, $Z = 1.96$, a margin of error $e = 0.05$, and the conservative choice $p = 0.5$, the large-population estimate is

$$n_0 = \frac{Z^2 p (1 - p)}{e^2} \approx 384.16.$$

Because our population is finite, we apply the finite population correction and obtain

$$n = \frac{n_0}{1 + \frac{n_0 - 1}{N}} \approx 381.$$

Accordingly, we annotate a stratified random sample of $n = 381$ files.

Strata correspond to files containing LLM inference call. Let N_i denote the number of eligible files in repository i . The allocation for repository i is computed proportionally as

$$n_i = n \cdot \frac{N_i}{N},$$

followed by standard rounding and a small adjustment to ensure $\sum_i n_i = n$. Within each repository, files are selected uniformly at random. Post-hoc checks show that per-smell frequencies deviate by at most 2.3 percentage points from the population, supporting the representativeness of the reduced labeling effort.

Because no public ground truth covering these LLM code smells currently exists, we constructed an adjudicated dataset through manual labeling by seven annotators. The annotator pool comprised the two first authors, both PhD students in software engineering whose research focuses on AI-based software systems, LLM-integrating systems, code smells, and static analysis, one Master’s-level student with two years of hands-on AI development experience and prior familiarity with the AI and ML code smells of Zhang et al. [7], and five additional contributors working in software engineering, including two with direct AI industry experience and two with general software engineering experience. Overall, the annotators had, on average, 3.4 years of professional experience. To avoid confirmation bias, none of the four annotators had prior knowledge of our detection tool *SpecDetect4LLM* or its outputs, and the labeling was conducted blind to tool predictions.

All annotators followed a structured protocol adapted from prior work [36, 38]. The protocol was documented in the replication package and included, for each smell, a definition, operational cues describing how the smell manifests in code, and illustrative examples clarifying borderline cases. To standardize the process and reduce transcription errors, annotators performed labeling through a dedicated web interface that supports file browsing and line-level marking of smell occurrences. The workload was distributed evenly, with each annotator labeling 55 files, for a total of 385 labeling assignments, which covers the full sample and supports overlap for agreement estimation.

Blind multi-annotator labeling followed by adjudication yielded strong inter-rater reliability, with Fleiss’ $\kappa = 0.79$ and per-smell agreement ranging from 0.72 to 0.92, indicating near-perfect agreement. Throughout the process, annotators were blind to *SpecDetect4LLM*’s implementation, internal smells, and any tool-produced outputs, which reduces the risk of confirmation bias. To support auditability and reuse, we release the full guidelines, code templates, and raw labels in the replication package [30].

7.4 Evaluation Metrics

We compare the approach’s detections (D) with the ground-truth set (G). Table 3 summarises the four outcomes; V denotes validated positives, V^c validated negatives.

As summarized in Table 3, *precision* is the proportion of validated positives among all detections $P = \frac{TP}{D}$, *recall* is the proportion of validated positives among all ground-truth instances $R = \frac{TP}{TP+FN}$, and *F₁-score* is the harmonic mean of the two $F_1\text{-score} = \frac{2 \times P \times R}{P+R}$.

Table 3 Outcome categories (TP: True Positives, FP: False Positives, FN: False Negatives, TN: True Negatives). V : smell instances, V^c : clean instances.

Reported	V (smell)	V^c (clean)
D (detected)	$TP = D \cap V$	$FP = D \cap V^c$
D^c (missed)	$FN = D^c \cap V$	$TN = D^c \cap V^c$

Table 4 RQ1 per smell confusion outcomes and accuracy metrics on the annotated set. For RENES and AIC, recall is not reported because the annotated set contains too few positives to support a stable estimate. In these cases, TP and FP are obtained through a manual audit of flagged instances.

Smell	TP	FP	FN	TN	Precision	Recall	F1
NMVP	84	13	14	121	0.866	0.857	0.862
TNES	33	9	15	175	0.786	0.688	0.733
NSO	26	4	8	194	0.867	0.765	0.812
UMM	18	5	14	195	0.783	0.562	0.655
NSM	16	0	1	215	1.000	0.941	0.970
RVP	3	0	2	227	1.000	0.600	0.750
OSP	3	0	1	228	1.000	0.750	0.857
RENES	0	0	N/A	N/A	0.000	N/A	N/A
AIC	0	0	N/A	N/A	0.000	N/A	N/A

Table 5 RQ1 aggregated metrics across smells on the annotated set.

Aggregation	Precision	Recall	F1
Micro average	0.855	0.769	0.810
Macro average	0.900	0.738	0.806

7.5 Results

In this section, we study and answer the research questions.

7.5.1 RQ₁: How precise is *SpecDetect4LLM* to detect LLM code smells?

This section evaluates the correctness of *SpecDetect4LLM* when detecting LLM code smells in real world projects. We compare tool predictions against our manually labeled ground truth on the annotated subset. For each smell, we compute standard confusion outcomes. True positives correspond to smell instances correctly reported by the tool. False positives correspond to reported instances that are not confirmed by manual

inspection. False negatives correspond to manually confirmed instances that are missed by the tool. True negatives correspond to non smell instances correctly left unflagged. From these counts, we derive precision, recall, and the F_1 -score per smell.

Table 3 summarises the smell-level results. We report both micro- and macro-averaged metrics to account for the imbalance between frequent and rare smells. Micro averaging pools all true positives, false positives, and false negatives across smells before computing precision, recall, and F_1 -score. It is therefore mainly influenced by frequent smells. Macro averaging first computes the metric independently for each smell and then averages the smell-level values, giving the same weight to each smell regardless of its frequency. Across smells, *SpecDetect4LLM* achieves a micro-averaged precision of 0.855, a micro-averaged recall of 0.769, and a micro-averaged F_1 -score of 0.810 (Table 5). The macro-averaged precision is higher at 0.900, while macro-averaged recall is slightly lower at 0.738, yielding a macro-averaged F_1 -score of 0.806. This divergence is consistent with a performance profile where some less frequent smells exhibit very high alert correctness, which increases macro precision, while their incomplete coverage depresses macro recall. Conversely, micro averaging is more influenced by the frequent smells in the annotated set, particularly NMVP, TNES, UMM, and NSO, whose precision and recall are high but not perfect.

At the smell level, NMVP and NSO show strong and relatively balanced behavior, with F_1 -score values of 0.862 and 0.812 respectively, indicating that most tool alerts correspond to true issues and that a large fraction of manually confirmed instances are recovered. TNES remains effective but exhibits residual error, with 9 false positives and 15 false negatives. UMM follows a similar pattern, with 5 false positives and 14 false negatives. NSM displays perfect precision but lower recall (0.941), suggesting that when the analyser flags a missing system message it is reliable, yet many true instances are missed. RVP and OSP achieve perfect precision with moderate recall, which is consistent with locally explicit cues that are easy to validate when present, but not always exposed in the syntactic forms captured by an intra file smell.

The error surface is largely explained by the observability limits of intra file static detection, a limitation inherited from *SpecDetect4AI* and deliberately preserved in *SpecDetect4LLM* to maintain scalability over large corpora. Concretely, the analyser reasons over local AST level evidence and does not reconstruct inter file data flow, interprocedural control flow, or runtime configuration resolution. As a result, false positives may arise when a call site appears underspecified in isolation although the constraint is enforced elsewhere, for instance through a shared wrapper that injects defaults or validates outputs. False negatives may arise when the relevant evidence is implemented through indirection, encapsulated in another module, or expressed through framework abstractions that hide the underlying provider call.

The following listings illustrate representative patterns that were encountered during manual adjudication and that explain how false positives and false negatives can be produced by an intra file analyser. These cases are not artefacts of our implementation but stem from the same design constraints and tradeoffs as *SpecDetect4AI*, on which *SpecDetect4LLM* is directly based.

Listing 10 exemplifies a typical cross module configuration pattern that challenges intra file reasoning. The call site in `app/chat.py` does not expose any `temperature`

argument and is therefore indistinguishable, locally, from a truly underspecified call. However, the project centralises the policy in `app/llm_client.py`, where the client is constructed with defaults that enforce the sampling configuration globally. Under an intra file AST model, this yields a natural false positive for TNES because the evidence needed to validate compliance is external to the analysed unit. Eliminating such cases requires inter file data flow reconstruction and interprocedural reasoning about how configuration and client instances propagate to call sites, which is deliberately out of scope for a scalable static analyser.

Listing 11 pushes this limitation further by introducing runtime selection of the concrete client implementation. The handler in `app/handler.py` invokes `get_llm()` and then performs a completion without explicitly setting `timeout` and `max_tokens`. Yet the factory in `app/factory.py` can return either a bounded wrapper or a raw client depending on a runtime flag derived from the environment. This means that the boundedness property is not a syntactic attribute of the call site, nor even a fixed attribute of the program under a single static interpretation. It is a property of a particular execution configuration. Consequently, an intra file static analyser cannot soundly decide whether the call will be bounded at runtime, and must approximate. In practice, this approximation can materialise as a false positive when the bounded client is selected, and as a true positive when the unbounded path is active.

To further explain this interpretation, we manually classified the residual FPs and FNs according to the type of information that is not available under the current intra file observation model. Among the 86 errors, 60 are related to inter file data flow or local value propagation that the analyser cannot fully reconstruct, including cases where relevant arguments are passed through variables, dictionaries, wrappers, helper abstractions, or client instances. Another 14 errors are related to runtime configuration resolution or cross module configuration layers, where the analysed call site appears underspecified although the missing constraint is introduced elsewhere in the project. Finally, 12 errors are related to interprocedural control flow, where the correctness of the detection depends on execution paths, conditional construction patterns, dynamic dispatch, or dependency injection. This distribution supports our interpretation that most remaining errors stem from the restricted observation model of the analyser rather than from inconsistencies in the smell definitions.

These examples motivate the evaluation stance adopted in RQ1. *SpecDetect4LLM* is a static, intra file analyser whose primary objective is to provide scalable and reliable warnings over large corpora. In this setting, maximising precision is often preferable to aggressively pursuing recall because warnings are meant to be actionable and triaged by developers, and excessive noise can undermine adoption. We therefore design smell detection rules to be conservative with respect to locally observable evidence, aiming to be *soundy* [39] within the intra file observation model, in the sense that reported violations are supported by explicit syntactic evidence at the analysed call site. Achieving system level soundness, in the strict sense, would require whole program reasoning that accounts for cross module configuration, dynamic dispatch, dependency injection, and runtime inputs such as environment variables and remote policies. Such an approach would substantially increase analysis cost and complexity, and it would still face undecidability and modelling gaps in dynamic Python ecosystems.

```

1 # file: app/chat.py
2 from app.llm_client import client
3
4 def reply(prompt):
5     return client.chat.completions.create(
6         model="gpt-4o",
7         messages=[{"role": "user", "content": prompt}],
8         # flagged as TNES because temperature is absent at the call site
9     )
10
11 # file: app/llm_client.py
12 from openai import OpenAI
13 from app.config import settings
14
15 client = OpenAI()
16
17 client = OpenAI(
18     # temperature is enforced centrally and not visible at the call site
19     default_headers={"x-temperature": str(settings.temperature)}
20 )

```

Listing 10 TNES false positive through cross module configuration injection

```

1 # file: app/handler.py
2 from app.factory import get_llm
3
4 def answer(prompt):
5     llm = get_llm()
6     return llm.chat.completions.create(
7         model="gpt-4o",
8         messages=[{"role": "user", "content": prompt}],
9         # flagged as UMM because timeout and max_tokens are not visible here
10    )
11
12 # file: app/factory.py
13 import os
14 from app.bounded_client import BoundedOpenAI
15 from openai import OpenAI
16
17 def get_llm():
18     # runtime switch, cannot be decided reliably intra file
19     if os.getenv("LLM_BOUNDED_MODE", "1") == "1":
20         return BoundedOpenAI()
21     return OpenAI()
22
23 # file: app/bounded_client.py
24 import os
25 from openai import OpenAI
26
27 class BoundedOpenAI(OpenAI):
28     def __init__(self):
29         super().__init__()
30         self._timeout_s = float(os.getenv("LLM_TIMEOUT_S", "15.0"))
31         self._max_tokens = int(os.getenv("LLM_MAX_TOKENS", "512"))
32
33     class chat:
34         class completions:
35             @staticmethod
36             def create(*args, **kwargs):
37                 timeout_s = float(os.getenv("LLM_TIMEOUT_S", "15.0"))
38                 max_tokens = int(os.getenv("LLM_MAX_TOKENS", "512"))
39
40                 kwargs.setdefault("timeout", timeout_s)
41                 kwargs.setdefault("max_tokens", max_tokens)
42
43                 client = OpenAI()
44                 return client.chat.completions.create(*args, **kwargs)

```

Listing 11 UMM false positive because bounds are injected through a runtime selected client implementation in another module

Precision oriented validation for low prevalence smells (RENES and AIC)

In our annotated subset, *RENES* and *AIC* occur too rarely to support a statistically meaningful recall estimate under random file sampling. In such sparse regimes, recall becomes either undefined or dominated by sampling noise, which would make any confusion matrix analysis for these smells uninformative. We therefore adopted a precision oriented protocol that directly evaluates the correctness of alerts conditional on the analyser emitting a warning.

Concretely, we manually audited the complete set of instances reported by *SpecDetectLLM* for *RENES* and *AIC* and adjudicated each alert as a true positive or false positive using the same operational decision sheet as in the main manual study. All audited alerts for these low prevalence smells were confirmed as true positives, with no false positives observed in the flagged set. This result supports the practical utility of these smells for practitioners, because when prevalence is low the primary question is whether emitted warnings can be trusted, rather than whether the analyser can recover an unobserved population of positives.

To mitigate the threat that an apparent absence of false positives is an artefact of narrow corpus patterns, we complemented the audit with smell level regression tests that explicitly encode both positive and negative scenarios. For *RENES*, the tests cover multiple providers and model families where reasoning controls are exposed through different API surfaces, including OpenAI reasoning oriented models, Anthropic models with explicit thinking configuration, and Gemini models with thinking related generation settings. For *AIC*, the tests capture anonymized or dynamically routed inference calls in which the effective model identity is not statically recoverable, as well as representative true negative controls in which the model is explicitly pinned to a concrete version. Importantly, both suites include false positive guards, such as non reasoning model calls that omit any reasoning configuration for *RENES*, and non LLM objects exposing a `create` method or wrapper based clients that hardcode a version pinned model for *AIC*. In combination, the manual audit provides external validity on real project instances, while the regression tests strengthen internal validity by constraining the smell semantics and preventing future changes to the analyser from reintroducing spurious matches.

This precision oriented validation does not claim that *RENES* and *AIC* have high recall in the wild. Instead, it establishes that when the analyser reports these low prevalence smells, the warning corresponds to a genuine integration issue under our specification. Further, by explicitly linking the regression tests to concrete positive and negative code patterns, we ensure that future tool revisions preserve the intended semantics and avoid reintroducing known sources of false positives.

Answer to RQ₁:

SpecDetect4LLM is precise at detecting LLM code smells. Aggregating all smells, the analyser achieves a micro averaged precision of 0.855 and a macro averaged precision of 0.900, indicating that reported warnings are correct in the large majority of cases. At the smell level, precision remains consistently high across the main smells, reaching 0.866 for NMVP, 0.867 for NSO, 0.783 for UMM, and 0.786 for TNES, while NSM, RVP, and OSP exhibit perfect precision in this sample. The remaining false positives are largely explained by the intra file static observation model inherited from *SpecDetect4AI*, where call sites can appear underspecified even though constraints are enforced through cross module wrappers, factories, or runtime configuration. In line with a soundy design objective, *SpecDetect4LLM* prioritises conservative, locally justified findings to keep warnings actionable and to limit noise during developer triage. For low prevalence smells such as RENES and AIC, the annotated set contains too few positives to support stable recall estimation, so RQ1 conclusions focus on the **high precision of the smells that appear with sufficient frequency** in the adjudicated ground truth.

7.5.2 RQ₂: Does *SpecDetect4LLM* scale efficiently across diverse LLM-integrating systems, regardless of system size?

We evaluate the runtime performance and scalability of *SpecDetect4LLM* by applying it to our full corpus of **171,194** Python files mined from Github repositories. Because the analyser is static, its computational cost is primarily driven by parsing and AST traversal within each module, rather than by global program structure. We therefore report both corpus level throughput and the distribution of per file runtimes to characterise scalability across diverse systems and file sizes.

Table 6 General Statistics of the Analysis

Metric	Value
Number of .py files analysed	171,194
Total analysis time	36,746.9 s (10h 12min 27s)
Throughput	4.66 files/s
Mean time per file	0.215 s
Median time per file	0.0222 s
95 th -percentile time per file	0.681 s
Minimum time per file	7.77×10^{-5} s
Maximum time per file	1,607.1 s

The complete run required **36,746.9 s** (10h 12min 27s), corresponding to an overall throughput of **4.66 files/s**. This throughput indicates that the analysis remains feasible at corpus scale, and it also provides a practical upper bound for expected wall clock time when processing new snapshots of similar magnitude. All experiments were executed on a MacBook Pro equipped with an Apple M4 Pro processor and **24 GB**

of RAM. Since analysis is embarrassingly parallel across files, this throughput can be further increased through parallel execution on multi core machines, while preserving identical detection semantics.

Runtime is highly concentrated on a small tail of difficult inputs. The median file is processed in **0.0222s**, and **95%** of files finish within **0.681s**, showing that for most modules the detector runs in the millisecond range. The mean is substantially higher at **0.215s**, which is consistent with a heavy tailed distribution where a small fraction of files dominates the total cost. Figure 5 visualises this skew using a log scaled summary plot.

This distributional profile is important for interpreting scalability with respect to system size. In large repositories, most files are typically small to medium sized, and their cost remains close to the median. Therefore, total runtime tends to grow approximately linearly with the number of files, with limited sensitivity to the overall architectural complexity of the system. In other words, repository scale affects wall clock time mainly through file count and the presence of exceptionally large modules, rather than through cross module interactions, which the tool intentionally does not model.

To better understand the sources of extreme runtimes, we manually inspected a sample of the slowest files. The maximum observed runtime was **1,607.1s**, and many other outliers correspond to very large test suites or vendored third party code, including substantial upstream libraries embedded in repositories. These cases are characterised by unusually large ASTs, deep nesting, and extensive use of metaprogramming patterns that increase parse and traversal cost. This evidence suggests that runtime variability is driven primarily by AST complexity and Python parsing overhead rather than by typical application code paths. From an engineering standpoint, these outliers motivate practical mitigations when the tool is used operationally, such as excluding vendor and generated code directories, limiting analysis to first party modules, or enforcing per file timeouts to prevent a small number of extreme inputs from delaying a full run.

Overall, the measured runtime behavior aligns with the scalability expectations of an intra file static detector. The strong right skew indicates that *SpecDetect4LLM* is fast on the vast majority of files encountered in heterogeneous LLM integrating projects, while a small set of exceptionally large or complex modules dominates the tail. This profile supports efficient large scale analysis across systems of varying size and structure, and it provides clear operational guidance for controlling worst case costs through common repository.

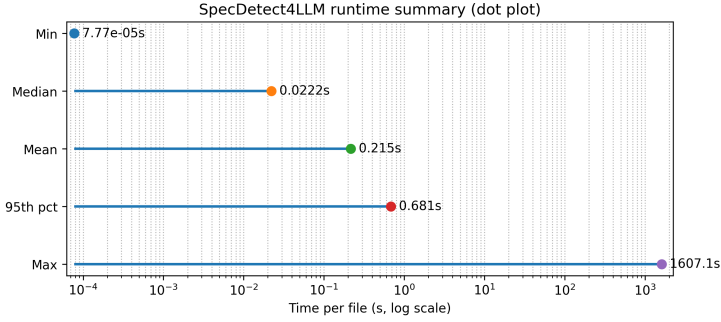


Fig. 5 Runtime summary per file on a log scale using a dot plot with stems (min, median, mean, 95th percentile, max).

Answer to RQ_2 :

***SpecDetect4LLM* scales efficiently across diverse LLM integrating systems.** On 171,194 Python files, it sustains a throughput of 4.66 files/s and processes the typical module in 0.0222s, with 95% of files completing within 0.681s. The remaining cost is concentrated in a small heavy tail of exceptionally large or syntactically complex files, most often originating from test suites or vendored third party code, indicating that runtime is governed by file level AST complexity and parsing overhead rather than by system level architectural size or cross module dependencies.

7.6 RQ_3 : How prevalent are LLM code smells in open-source software systems that integrate LLMs, and which LLM code smells occur most frequently?

Having established the accuracy and scalability of *SpecDetect4LLM*, we next examine how LLM code smells manifest in practice across the studied corpus.

To answer this question, we analyze the prevalence of each LLM code smell using complementary frequency indicators, namely the number of occurrences, affected files, and affected projects. This analysis allows us to identify the smells that are most prominent in practice and to derive empirical insights into their distribution across open source systems.

7.7 Dataset and prevalence metrics

We conducted a large scale prevalence study by running *SpecDetect4LLM* on our entire dataset of 692 LLM-integrating systems comprising 171,194 source files. We report prevalence at two complementary levels of granularity. File level prevalence measures the proportion of files that contain at least one instance of a smell. Project level prevalence measures the proportion of projects in which at least one file contains

Table 7 Prevalence per rule across 171,194 files and 692 projects. File and project prevalence are computed against the full dataset totals.

Smell	Files	File prev. (%)	Projects	Project prev. (%)	Occurrences
NMVP	5,543	3.24	381	55.06	15,100
TNES	1,859	1.09	304	43.93	4,219
NSO	1,524	0.89	302	43.64	3,495
UMM	1,425	0.83	291	42.05	3,372
NSM	761	0.44	175	25.29	1,665
RVP	302	0.18	131	18.93	441
OSP	183	0.11	93	13.44	302
RENES	6	0.00	6	0.87	15
AIC	7	0.00	5	0.72	8

an instance of a smell. In addition, we report the number of occurrences, which corresponds to the number of alerts emitted by the detector and captures intensity beyond simple presence.

7.8 Overall prevalence

Across the full dataset, 7,388 files are flagged by at least one rule, which corresponds to a file level prevalence of 4.32% (7,388 out of 171,194). At the project level, 509 projects contain at least one alert, yielding a project level prevalence of 73.55% (509 out of 692). These results highlight a clear contrast between breadth and locality. Smells are widespread across systems, yet typically concentrated in a small subset of files per project, consistent with the fact that LLM integration logic is usually implemented in dedicated modules or pipeline components rather than scattered uniformly throughout the codebase.

From an intensity perspective, we observe 28,617 occurrences overall, with an average of 3.87 occurrences per flagged file (28,617/7,388) and 56.22 occurrences per affected project (28,617/ 509). The median affected project, however, has 12 occurrences and only 4 flagged files, indicating a strongly right skewed distribution in which a minority of projects concentrate a large share of alerts.

7.9 Per rule prevalence and dominant patterns

Table 7 summarises prevalence per rule and Figure 6 ranks rules by project level prevalence. Beyond raw frequency, each smell exhibits a characteristic mechanism that explains why it tends to appear, where it concentrates in code, and what risks it creates for LLM-integrating systems.

NMVP, LLM Version Pinning Not Explicitly Set.

NMVP is the dominant smell, affecting 55.06% of projects (381 out of 692) and 3.24% of files (5,543 out of 171,194), and accounting for 52.77% of all occurrences (15,100 out of 28,617). This prevalence is consistent with an engineering practice where developers pass a high level model alias (for example a family name) rather than a pinned

model identifier or revision. The key particularity of NMVP is that it directly threatens reproducibility and auditability. When the provider updates or reroutes an alias, the same code can yield different outputs without any change in the repository history, which complicates debugging, regression testing, and safety reviews. The high occurrence density (2.72 occurrences per touched file on average) suggests that once a project adopts non pinned calls, the pattern propagates across wrappers and utilities, producing repeated instantiations of the same fragility. A practical implication is that pinning should be treated as part of the call contract and enforced at the boundary layer, typically through a central client factory that accepts only pinned identifiers and rejects aliases during CI.

TNES, LLM Temperature Not Explicitly Set.

TNES affects 43.93% of projects (304 out of 692) but only 1.09% of files (1,859 out of 171,194), with 4,219 occurrences. The defining trait of TNES is that it silently delegates a core stochasticity control to provider defaults, which are both model specific and subject to change. This smell is particularly likely to occur in early prototypes and research code, where teams focus on prompt engineering and treat sampling as an implementation detail. In production settings, TNES undermines stability because output variance can change across environments, across SDK versions, or across provider side updates. It also weakens evaluation because experimental comparisons can become partially confounded by implicit sampling. The observed pattern, high project prevalence with low file prevalence, indicates that temperature omission is concentrated in a small number of integration files, often a shared helper or wrapper. This makes remediation tractable. Introducing an explicit temperature policy, including a deterministic default for automation and an exploratory default for creative tasks, can reduce variance while preserving intentional diversity when needed.

UMM, LLM Calls Without Bounded Metrics.

UMM appears in 42.05% of projects (291 out of 692) with 3,372 occurrences. UMM is distinctive because it is an operational risk smell rather than a purely functional one. Unbounded calls typically omit constraints such as maximum tokens, timeouts, retry budgets, or cost ceilings, thereby exposing systems to latency spikes, runaway costs, and cascading failures under load. In practice, this smell often emerges when the call site is written as a direct SDK invocation without passing guardrail parameters, assuming that infrastructure will handle stability. The prevalence suggests that many projects still do not encode resource governance at the LLM boundary. A key recommendation is to centralise budget and latency constraints in a single gateway wrapper that applies strict defaults and logs budgets per request, since these constraints are difficult to enforce reliably when scattered across call sites.

NSO, No Structured Output in Pipeline.

NSO affects 43.64% of projects (302 out of 692) with 3,495 occurrences. NSO is characterised by treating LLM responses as free form text even when the downstream pipeline expects a parseable or schema compliant artefact. This smell is especially salient in LLM based automation, where unstructured outputs create brittle parsing

heuristics, increase error handling complexity, and amplify silent failure modes. Its co-prevalence with TNES and UMM is not accidental. Systems that treat the LLM call as an informal component often also omit explicit sampling and resource contracts. The most effective mitigation is to adopt a typed interface at the integration boundary, such as JSON schema constraints, response models, or constrained decoding, which reduces ambiguity and improves testability by enabling deterministic validators for output shape.

NSM, LLM With No System Message.

NSM affects 25.29% of projects (175 out of 692) with 1,665 occurrences. NSM is conceptually different from the configuration oriented smells above because it targets instruction hierarchy and behavioral alignment. Omitting a system message can lead to weak global constraints, inconsistent tone, and increased susceptibility to prompt injection when user content is untrusted or externally sourced. This smell is also sensitive to API style. Some frameworks encourage system prompts, while others embed global instructions in templates or middleware. Hence, a portion of the observed prevalence likely reflects genuine omissions at the call site, while another portion may correspond to projects that rely on hidden system layers that intra file analysis cannot observe. Practically, teams should treat system level policy as a first class artefact, store it in version control, and ensure that all user driven interactions are mediated by a stable instruction scaffold.

RVP, Raw Vision Payload.

RVP affects 18.93% of projects (131 out of 692) with 441 occurrences. RVP typically arises when vision requests embed raw images, often as large base64 payloads or unprocessed originals, directly into prompts. The particularity of RVP is that it couples security, privacy, and cost risks. Raw payloads can leak sensitive metadata or unnecessary content, and large images inflate tokenisation and inference cost while increasing latency. The non trivial prevalence indicates that multimodal usage is sufficiently common that payload hygiene is now a practical concern. Mitigation is largely engineering oriented. Preprocess images through resizing, compression, redaction, and optional feature extraction, then pass the minimal representation needed for the task, while enforcing explicit size limits at the gateway.

OSP, Overspecified Sampling Parameters.

OSP affects 13.44% of projects (93 out of 692) with 302 occurrences. OSP is characterised by specifying multiple sampling controls simultaneously, such as temperature together with top-p and additional penalties, without a clear rationale or calibration. The risk is not merely redundancy. Over specification can yield unstable or unintuitive distributions, complicate reproducibility, and make behavior difficult to debug because small parameter shifts interact non linearly. The moderate prevalence suggests that some teams adopt parameter bundles copied from examples or tutorials rather than tuning parameters for a task. A principled mitigation is to standardise a minimal sampling policy, specify only the parameters necessary for the intended behavior, and document the rationale for each non default choice in the wrapper layer.

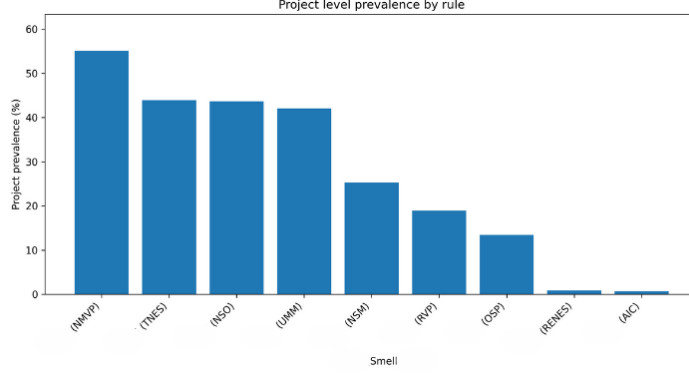


Fig. 6 Project level prevalence by rule. Values correspond to the fraction of projects (out of 692) in which each smell is triggered at least once.

RENES, Reasoning Effort Not Explicitly Set and AIC, Anonymous Inference Call.

RENES and AIC are rare in this corpus, affecting 0.87% (6 out of 692) and 0.72% (5 out of 692) of projects, respectively. For RENES, rarity can reflect limited adoption of provider specific reasoning effort controls, or the fact that such controls are often configured outside the call site, for example via environment driven configuration, which reduces detector observability. For AIC, low prevalence is consistent with the widespread use of wrappers and SDK clients that encapsulate provider identity, model routing, or authentication, meaning that anonymity may occur at a higher abstraction layer than the immediate call site. These long tail results should therefore be interpreted cautiously. Low measured prevalence may indicate genuine rarity, reduced static observability, or both.

Relating prevalence to the taxonomy.

To connect the prevalence results to the taxonomy (Section 5), we interpret each smell through its defect category, namely Structural or API usage, Data semantics, and Protocol. This perspective reveals a clear skew toward protocol-related defects, which capture missing explicitness in system-wide LLM invocation contracts such as model stability, sampling control, and resource bounds. Concretely, when aggregating occurrences by category, protocol smells (NMVP, TNES, UMM, RENES) account for 22,706 out of 28,617 alerts (79.3%), while data-semantics smells (NSO, RVP) account for 3,936 (13.8%) and structural or API-usage smells (NSM, OSP, AIC) account for 1,975 (6.9%). This distribution suggests that, in open-source LLM integrations, the dominant risks arise less from exotic API misuse and more from implicit defaults and unstable configurations that affect reproducibility and operational reliability across runs. It also provides a parsimonious explanation for why a small subset of smells dominates prevalence, as protocol-related choices tend to be shared across many call sites once adopted in a project-level wrapper or client factory.

7.10 Implications and actionable guidance

The prevalence results yield three actionable implications for engineering practice. First, the combination of high project level prevalence and low file level prevalence indicates that LLM related integration issues are widespread across systems yet structurally localised within codebases. In other words, smells tend to concentrate in a small set of dedicated modules that encapsulate model access, such as client factories, wrappers, and orchestration pipelines. This concentration suggests that remediation can be cost effective. Addressing a limited number of integration files can provide a disproportionate reduction in system wide risk, without requiring broad refactoring across unrelated components.

Second, the dominance of NMVP provides evidence that reproducibility and traceability remain systemic weaknesses in current LLM integration practice. When calls rely on non pinned model identifiers, system behavior can drift over time due to provider side updates, even when the application code remains unchanged. This undermines regression testing, incident analysis, and compliance oriented audits. Consequently, model identification should be treated as part of the stable interface of an LLM call, at the same level as prompt specification and output validation. Practically, teams can operationalise this requirement by enforcing pinned model identifiers through configuration schemas, code review checklists, and continuous integration guards, so that model upgrades become explicit, reviewable, and reversible changes.

Third, the high prevalence of the second tier smells, notably TNES, UMM, and NSO, indicates that reliability and operational robustness are recurring challenges once LLM functionality is introduced. These smells reflect missing explicitness at the call boundary, covering sampling control, resource and cost bounds, and the structure of returned outputs. A robust mitigation strategy is to standardise an integration contract that makes such parameters explicit and enforceable. In practice, a lightweight yet effective design is to centralise all provider calls within a single wrapper or client abstraction that applies explicit defaults, enforces bounded resources, logs critical parameters, and returns validated structured outputs when required. Restricting direct provider calls outside this abstraction reduces both defect introduction and maintenance overhead by consolidating integration policy in one auditable location.

Answer to RQ₃:

LLM code smells are widespread in open source systems, with a small set of smells dominating prevalence. Across 692 LLM integrating projects and 171,194 Python files, *SpecDetect4LLM* reports that the most prevalent smells are concentrated in underspecified call site practices, where configuration and interaction contracts are not made explicit at the invocation point. The prevalence distribution is strongly skewed: a few smells account for most affected files and projects, while the remaining smells appear only sporadically. This pattern suggests that open source LLM integrations are primarily challenged by missing explicitness and incomplete invocation level specification, rather than by niche provider specific features. At the same time, the long tail of rare smells should not be interpreted as irrelevance, as their low observed prevalence can reflect both genuinely uncommon practices and limited local observability when constraints are implemented through cross module wrappers, factories, or runtime configuration. Overall, RQ₃ indicates that prevalence is not uniform across smells, and that practitioners are most likely to encounter a recurring subset of high frequency smells that are broadly shared across projects, complemented by a sparse tail of context dependent smells.

8 Limitations and threats to validity

This section summarizes the main threats to validity. For clarity, we structure each validity dimension around three parts of the study, namely the catalog construction, the detection approach, and the prevalence study.

8.1 Construct validity

8.1.1 Catalog construction

A key threat is whether the proposed smells match real recurring problems in LLM integration. Because the catalog is built from mixed evidence sources, similar recommendations can sometimes refer to different issues depending on context. We mitigate this by keeping only smells that are observable in code and linked to a concrete remediation. Still, some practices are context dependent, so the same pattern may not always be harmful.

8.1.2 Detection

Some smells describe properties that may be enforced outside the local file, for example via wrappers or configuration layers. A static analyser can therefore flag a call that looks incomplete even if constraints are applied elsewhere, or miss cases where evidence is split across modules. We address this by defining conservative rules and by interpreting matches as local indicators rather than full system level guarantees. In addition, our tool is built on *SpecDetect4AI*, a state of the art static analyser for AI-specific code smells that was designed to be more precise and extensible, which

provides a strong and stable foundation for implementing and evolving LLM specific rules.

8.1.3 Prevalence

Prevalence is measured from what the rules can observe, which can bias counts up or down when a rule relies on a local proxy. We therefore interpret prevalence as observable prevalence under local static analysis, not as the true frequency of violations in the whole system.

8.2 Internal validity

8.2.1 Catalog construction

The catalog may be influenced by what is most documented, for example popular providers or frameworks, which could bias the set of smells. We reduce this by requiring converging evidence across source types and by defining smells in provider agnostic terms when possible.

8.2.2 Detection

Detection results can be affected by implementation choices and by code patterns that hide calls, such as aliasing or abstraction layers. We mitigate this through regression tests covering typical and tricky cases, and by manual auditing of flagged instances when needed. Some residual errors may remain due to the limits of lightweight static analysis.

8.2.3 Prevalence

Prevalence can be confounded by corpus composition, for instance large repositories, vendored code, or test suites that concentrate certain patterns. We mitigate this by reporting prevalence at multiple levels such as occurrences, files, and projects, and by interpreting results with the observed skew in mind.

8.3 Conclusion validity

8.3.1 Catalog construction

Evidence does not imply the same impact in all settings. We therefore present smells as risk factors and avoid framing them as universally harmful defects.

8.3.2 Detection

For rare smells, recall estimates from random sampling are unstable. We address this by adapting the evaluation, focusing on precision for low prevalence smells, and avoiding strong claims when the number of positives is too small.

8.3.3 Prevalence

Prevalence distributions are often highly skewed, so comparisons across smells can be sensitive to a few large projects. We report complementary indicators and draw cautious conclusions, especially for rare smells.

8.4 External validity

8.4.1 Catalog construction

The main external threat to validity is that results may not fully generalize beyond open source Python systems. Other languages and environments may implement LLM integration through different mechanisms. We keep definitions conceptual, but extensions will be needed in other ecosystems.

There is also the threat of temporal instability and potential obsolescence of individual smells. This threat is a direct consequence of the nature of the field, which is still being established and is in rapid evolution, creating instability in standards and practices. However, we argue that it is precisely because of this that such work is essential, so that guidelines can be established and standards may follow. This is also a recurring limitation in smell catalogs in software engineering more broadly, where catalogs are often tied to API versions and require periodic revision as ecosystems evolve. Furthermore, even when a newer API version or model deprecates a given LLM Code Smell, older API versions and models still continue to be used in existing systems, meaning the smells remain applicable on these codebases.

Additionally, the possible future obsolescence of an LLM code smell does not imply that the principle behind it becomes obsolete with it. We argue that the core ideas and principles motivating the smells remain relevant as the field of LLMs matures, even if the way some concepts are applied may evolve. Therefore, this work positions itself as an initial foundation to be built upon as the field matures, rather than a fixed specification.

It is also important to note that not all smells are equally exposed to this risk. In general, Structural and Data Semantic LLM code smells are less at risk compared to Protocol ones, which are more directly dependent on provider API designs. Even within Protocol smells, the risk is not uniform. Smells such as TNES and RENES, for instance, are already subject to some variation across providers and models, and are therefore more susceptible than others.

8.4.2 Detection

A threat to validity comes from the fact that the current *SpecDetect4LLM* tool is based on lightweight AST-only static analysis. This design is effective for locally explicit manifestations, but it becomes more limited when the relevant evidence is distributed across files, procedures, wrappers, or configuration artefacts. LLM code smells may therefore depend on non-local relations that cannot be reconstructed within the current observation model, including inter-file, control-flow, and data-flow dependencies. As a result, the analyser may produce false positives when a call appears underspecified in isolation although the relevant constraint is enforced elsewhere, and false negatives

when the decisive occurrences are implemented through indirection or resolved only at runtime. We mitigate this threat by defining conservative rules and by interpreting detections as locally observable indicators rather than full system-level guarantees. Future work will explore richer forms of static reasoning to improve coverage for such non-local manifestations. **Article pas seulement l'outil + precision/recall + valeurs recall + precision**

Our detection tool SpecDetect4LLM also inherits the obsolescence risks of the smells it detects, as their concrete manifestations may change with API versions. However, SpecDetect4LLM's DSL and rule-based nature, inherited from SpecDetect4AI, not only makes it extensible but also easily maintainable and adjustable over time, substantially reducing obsolescence concerns at the detection level.

8.4.3 Prevalence

Observed prevalence in open source may differ from industrial or regulated settings, where practices and encapsulation differ. We therefore interpret prevalence as trends in our corpus and encourage replications in other contexts.

9 Related Work

9.1 Domain-Specific Code Smells

The concept of **code smells** is well established in software engineering [6]. Over time, researchers have extended the notion by developing catalogs and taxonomies for specific application domains, such as for Android applications [40] and Machine Learning (ML) [7]. We essentially follow the same approach, but for LLM-integrating systems. Therefore, our LLM code smells distinguish themselves by their application domain specific to LLM inference inside source code.

Mahmoudi et al. introduced *SpecDetect4AI* [11], a static source code analysis tool for the detection of AI-specific code smells [7]. As mentioned in section 6, *SpecDetect4LLM* is derived from this foundation by incorporating a new set of DSL-defined detection rules for LLM code smells. We selected *SpecDetect4AI* as the base analyser because, in prior evaluations, it outperformed existing tools from the literature in terms of accuracy, and because its DSL-based rule layer is explicitly designed for extensibility, enabling the systematic addition of new detection rules without reengineering the underlying analysis pipeline. While both tools share the same architecture and workflow, they address distinct application domains.

9.2 LLM Integration Defects

In our previous work [10], we introduced the concept of LLM code smells, presented an initial catalog of five defects, and provided the first version of *SpecDetect4LLM*. However, that study was limited to preliminary precision measurements and a prevalence study on 200 open-source projects. This paper consolidates and extends that work by expanding the catalog to nine defects, introducing a lightweight taxonomy, extending *SpecDetect4LLM*, and presenting a comprehensive evaluation of detection

effectiveness (precision and recall on 288 source files) and prevalence (on a corpus of 692 projects).

The closest existing works to our study, with the exception of our previous work, are the taxonomy of code defects for LLM-based autonomous agents and the taxonomy of defects for LLM-enabled and RAG systems, introduced by **Ning et al.** [41] and **Shao et al.** [2] respectively.

While some defects introduced by **Ning et al.** [41], such as *LLM API-related Defect*, relate to similar integration concerns, their taxonomy differs from ours in scope and granularity. Ning et al. focus on broader agent-level defects, whereas our work targets recurrent source-code practices at LLM inference call sites, which may appear both in simple integrations and agentic systems [42]. Similarly, the defects studied by **Shao et al.** [2] describe higher-level symptoms in LLM-enabled and RAG systems, rather than concrete coding practices. In contrast, our LLM code smells identify implementation-level practices that undermine maintainability, reliability, performance, or robustness without necessarily constituting direct failures.

Shao et al.'s [2] catalog of defects is based on general failure modes, **Ning et al.**'s [41] catalog contains mainly agent workflow issues, while our LLM code smells present concrete implementation practices at the inference call site that should be avoided, like an implicit contract that should be respected. While there is some overlap between the integration defects found in both **Ning et al.** and **Shao et al.**, it remains limited.

The biggest overlap for both is with NSO. In **Shao et al.**, it relates to *Incompatible LLM output format*, which describes downstream components receiving LLM responses that do not match the expected structure. In **Ning et al.**, NSO overlaps with *LLM Output Parsing Error*, which describes the absence of error-handling or flexible parsing mechanisms when parsing model outputs. Both target the same risk as NSO, a mismatch between the expected structure and the free-form aspect of LLM outputs. The difference is that these two defects describe the resulting failure, while NSO targets the concrete coding practice that allows the failure to occur in the first place.

For **Shao et al.** [2] there is also partial overlap with UMM and NSM, UMM partially overlaps with *Exceeding LLM context limit*, which discusses prompts that surpass the maximum context window and cause truncation. The overlap is with the token-budget dimension of UMM, since *Exceeding LLM context limit* does not address timeout or retry configuration. It is also taken from a different angle, Shao et al.'s defect focuses on how truncation may happen when the context limit is reached, while we argue that an unspecified token budget may result in additional costs and delays. We therefore recommend for practitioners to choose a token budget tuned to the needs of the task given to the LLM. As for NSM, it overlaps very slightly with *Unclear context in prompt* and *Lacking restrictions in prompt*. While these integration defects relate more to prompt smells, which are out of the scope of this work, we recognize some overlap with the fact of missing a system message, as it is a useful mechanism to properly specify instructions given to an LLM.

The remaining LLM Code Smells, have no direct counterpart in either taxonomy. The full mapping is summarized in Table 8.

Table 8 LLM code smell catalog overlap with Shao et al. and Ning et al.

LLM Smell	Code	Shao et al.	Ning et al.
NMVP		-	-
TNES		-	-
NSO		Incompatible LLM output format	LLM Output Parsing Error
UMM		Exceeding LLM context limit	-
NSM		Unclear context in prompt; Lacking restriction in prompt	-
RVP		-	-
OSP		-	-
RENES		-	-
AIC		-	-

9.3 Related Concepts

Multiple similar, but fundamentally different concepts and works exist, these includes:

- **Taxonomies of failures** in LLM-Integrating systems that emphasize high-level symptoms rather than code-level practices [43–45];
- **Prompt-smell taxonomies** that are complementary to our study, but that operate on prompts, not code [8, 46];
- **Anti-patterns** for LLM inference **benchmarking** rather than integration [47];
- **Analyses of defects, anti-patterns, and code smells in LLM-generated code**, whereas we study human-written source code that integrate LLMs [48, 49].
- **Large Language Model Operations (LLMOps)**, a subtype of MLOps, addresses the operational lifecycle of LLM models (deployment, serving, monitoring) [50–53]. Our work is distinct, addressing the quality of source code that integrates LLMs, and therefore positioning itself within the software engineering practices related to the lifecycle of LLM-integrating systems.

10 Conclusion

In this work, we refine the concept of LLM code smells by extending our earlier preliminary work [10]. We construct a catalog of nine LLM code smells through a systematic literature review combined with grey literature mining, and organize this catalog into a taxonomy composed of three categories: Structural, Data-Semantic, and Protocol-Related. We also present *SpecDetect4LLM*, a static analysis tool for detecting LLM code smells, and evaluate its detection effectiveness in terms of precision and recall on 381 source code files. Finally, we complement this study with an empirical assessment of the prevalence of LLM code smells across 692 open-source software projects, totaling 171,194 source files.

Our results show that *SpecDetect4LLM* achieves a macro-averaged precision of 0.913 and a macro-averaged recall of 0.718, indicating effective detection. They further

reveal that 73.55% of the analyzed projects exhibit at least one LLM code smell, suggesting that such defects are widespread in LLM-integrating systems and reinforcing the relevance of a dedicated catalog and taxonomy.

References

- [1] Xia, Z., Zhu, L., Li, B., Chen, F., Li, Q., Liao, C., Wang, F., Liu, H.: Analyzing 16,193 LLM Papers for Fun and Profits (2025). <https://arxiv.org/abs/2504.08619>
- [2] Shao, Y., Huang, Y., Shen, J., Ma, L., Su, T., Wan, C.: Are LLMs Correctly Integrated into Software Systems? (2025). <https://arxiv.org/abs/2407.05138>
- [3] Khatun, A.: Uncovering the reliability and consistency of ai language models: A systematic study. PhD thesis, University of Waterloo (August 2024). <https://uwspace.uwaterloo.ca/items/e01e11a6-e033-4f6a-85c6-849fba74e039>
- [4] Yang, W., Some, L., Bain, M., Kang, B.: A comprehensive survey on integrating large language models with knowledge-based methods. *Knowledge-Based Systems* **318**, 113503 (2025) <https://doi.org/10.1016/j.knosys.2025.113503>
- [5] Bucaioni, A., Weyssow, M., He, J., Lyu, Y., Lo, D.: A Functional Software Reference Architecture for LLM-Integrated Systems (2025). <https://arxiv.org/abs/2501.12904>
- [6] Fowler, M., Beck, K., Brant, J., Opdyke, W., Roberts, D.: Refactoring: Improving the Design of Existing Code. Addison-Wesley Longman Publishing Co., Inc., USA (1999)
- [7] Zhang, H., Cruz, L., Deursen, A.: Code Smells for Machine Learning Applications (2022). <https://arxiv.org/abs/2203.13746>
- [8] Tian, H., Wang, C., Yang, B., Zhang, L., Liu, Y.: A Taxonomy of Prompt Defects in LLM Systems (2025). <https://arxiv.org/abs/2509.14404>
- [9] Paul, D.G., Zhu, H., Bayley, I.: Investigating the Smells of LLM Generated Code. SSRN. Available at SSRN (2025). <https://doi.org/10.2139/ssrn.5601126> . <https://ssrn.com/abstract=5601126>
- [10] Mahmoudi, B., Chenail-Larcher, Z., Moha, N., Stiévenart, Q., Avellaneda, F.: Specification and detection of LLM code smells. In: Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering, New Ideas and Emerging Results (ICSE-NIER '26). Association for Computing Machinery, New York, NY, USA (2026). <https://doi.org/10.1145/3786582.3786835> . <https://doi.org/10.1145/3786582.3786835>
- [11] Mahmoudi, B., Moha, N., Stievenert, Q., Avellaneda, F.: AI-Specific Code Smells: From Specification to Detection (2025). <https://doi.org/10.48550/arXiv.2509.>

- [12] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Proceedings of the 31st International Conference on Neural Information Processing Systems. NIPS'17, pp. 6000–6010. Curran Associates Inc., Red Hook, NY, USA (2017)
- [13] Zhang, J., Huang, J., Jin, S., Lu, S.: Vision-language models for vision tasks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **46**(8), 5625–5644 (2024) <https://doi.org/10.1109/TPAMI.2024.3369699>
- [14] OpenAI: Learning to Reason with LLMs. Technical report (2024). <https://openai.com/index/learning-to-reason-with-llms/>
- [15] ISO/IEC/IEEE: ISO/IEC/IEEE 24765:2017 Systems and software engineering: Vocabulary. International standard, International Organization for Standardization (2017)
- [16] ISO/IEC: ISO/IEC 25010:2023 Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): Product quality model. International standard, International Organization for Standardization (2023)
- [17] IEEE: IEEE Standard Glossary of Software Engineering Terminology. Ieee std 610.12-1990, Institute of Electrical and Electronics Engineers (1990). <https://doi.org/10.1109/IEEESTD.1990.101064>
- [18] Kitchenham, B., Charters, S.: Guidelines for performing systematic literature reviews in software engineering. Technical Report EBSE-2007-01, EBSE 2007 (2007). https://www.elsevier.com/_data/promis_misc/525444systematicreviewsguide.pdf
- [19] Cherief, H.A., Mahmoudi, B., Chenail-Larcher, Z., Moha, N., Sti'evenart, Q., Avellaneda, F.: An Automated Grey Literature Extraction Tool for Software Engineering (2025). <https://arxiv.org/abs/2512.23066>
- [20] Page, M.J., McKenzie, J.E., Bossuyt, P.M., Boutron, I., Hoffmann, T.C., Mulrow, C.D., Shamseer, L., Tetzlaff, J.M., Akl, E.A., Brennan, S.E., Chou, R., Glanville, J., Grimshaw, J.M., Hrobjartsson, A., Lalu, M.M., Li, T., Loder, E.W., Mayo-Wilson, E., McDonald, S., McGuinness, L.A., Stewart, L.A., Thomas, J., Tricco, A.C., Welch, V.A., Whiting, P., Moher, D.: The prisma 2020 statement: An updated guideline for reporting systematic reviews. *BMJ* **372**, 71 (2021) <https://doi.org/10.1136/bmj.n71>
- [21] Page, M.J., McKenzie, J.E., Bossuyt, P.M., Boutron, I., Hoffmann, T.C., Mulrow, C.D., Shamseer, L., Tetzlaff, J.M., Akl, E.A., Brennan, S.E., Chou, R., Glanville,

- J., Grimshaw, J.M., Hrobjartsson, A., Lalu, M.M., Li, T., Loder, E.W., Mayo-Wilson, E., McDonald, S., McGuinness, L.A., Stewart, L.A., Thomas, J., Tricco, A.C., Welch, V.A., Whiting, P., Moher, D.: Prisma 2020 explanation and elaboration: Updated guidance and exemplars for reporting systematic reviews. *BMJ* **372**, 160 (2021) <https://doi.org/10.1136/bmj.n160>
- [22] Kitchenham, B., Madeyski, L., Budgen, D.: Segress: Software engineering guidelines for reporting secondary studies. *IEEE Transactions on Software Engineering* **49**(3), 1273–1298 (2023) <https://doi.org/10.1109/TSE.2022.3174092>
- [23] Schardt, C., Adams, M.B., Owens, T., Keitz, S., Fontelo, P.: Utilization of the pico framework to improve searching pubmed for clinical questions. *BMC Medical Informatics and Decision Making* **7**, 16 (2007) <https://doi.org/10.1186/1472-6947-7-16>
- [24] Dybå, T., Dingsøyr, T., Hanssen, G.K.: Applying systematic reviews to diverse study types: An experience report. In: *Proceedings of the First International Symposium on Empirical Software Engineering and Measurement (ESEM 2007)*, pp. 225–234. IEEE, ??? (2007). <https://doi.org/10.1109/ESEM.2007.59> . <https://doi.org/10.1109/ESEM.2007.59>
- [25] Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J., Wen, J.: A survey of large language models. *arXiv preprint* (2023) arXiv:2303.18223 [cs.CL]
- [26] Garousi, V., Felderer, M., M"antyl"a, M.V.: Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and Software Technology* **106**, 101–121 (2019) <https://doi.org/10.1016/j.infsof.2018.09.006>
- [27] Kamei, F., Wiese, I., Pinto, G., Ribeiro, M., Soares, S.: On the use of grey literature: A survey with the brazilian software engineering research community. In: *Proceedings of the 34th Brazilian Symposium on Software Engineering. SBES '20. Association for Computing Machinery, ???* (2020). <https://doi.org/10.1145/3422392.3422442>
- [28] AI, P.: Perplexity <https://www.perplexity.ai/>. <https://www.perplexity.ai/> (2025)
- [29] Hugging Face: Hugging Face - The AI Community Building the Future. <https://huggingface.co/> Accessed 2025-09-25
- [30] Mahmoudi, B., Chenail Larcher, Z.: Replication_Package_LLM-code_smells. https://github.com/Brahim-Mahmoudi/Code_Smell_LLM (2025)
- [31] OpenAI: API Reference - Chat Completions (2025). <https://platform.openai.com/docs/api-reference/chat> Accessed 2025-09-25

- [32] Anthropic: Messages API - Claude Docs (2025). <https://docs.claude.com/en/api/messages> Accessed 2025-09-25
- [33] OpenAI: Images and Vision — OpenAI API Documentation (2025). <https://developers.openai.com/api/docs/guides/images-vision>
- [34] Anthropic: Vision - Claude API Documentation (2025). <https://platform.claude.com/docs/en/build-with-claude/vision>
- [35] OpenAI Developer Community: Clarifications on setting temperature = 0. Discussion thread accessed 2025-12-09 (2024). <https://community.openai.com/t/clarifications-on-setting-temperature-0/886447>
- [36] Nandani, H., Saad, M., Sharma, T.: DACOS: A manually annotated dataset of code smells. In: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1–12 (2023). <https://doi.org/10.1109/MSR59073.2023.00067>
- [37] Cochran, W.G.: Sampling Techniques, 3rd edn. John Wiley & Sons, New York (1977). Chap. 5
- [38] Passi, S., Jackson, S.J.: Trust in data science: Collaboration, translation, and accountability in corporate data science projects. *Proc. ACM Hum.-Comput. Interact.* **2**(CSCW) (2018) <https://doi.org/10.1145/3274405>
- [39] Livshits, B., Sridharan, M., Smaragdakis, Y., Lhoták, O., Amaral, J.N., Chang, B.-Y.E., Guyer, S.Z., Khedker, U.P., Møller, A., Vardoulakis, D.: In defense of soundness: a manifesto. *Commun. ACM* **58**(2), 44–46 (2015) <https://doi.org/10.1145/2644805>
- [40] Carvalho, S.G., Aniche, M., Veríssimo, J., Garcia, A., Alves, V., Gheyi, R.: An empirical catalog of code smells for the presentation layer of android apps. *Empirical Software Engineering* **24**(6), 3546–3586 (2019) <https://doi.org/10.1007/s10664-019-09768-9>
- [41] Ning, K., Chen, J., Zhang, J., Li, W., Wang, Z., Feng, Y., Zhang, W., Zheng, Z.: Defining and Detecting the Defects of the Large Language Model-based Autonomous Agents (2024). <https://arxiv.org/abs/2412.18371>
- [42] Ke, Z., Jiao, F., Ming, Y., Nguyen, X.-P., Xu, A., Long, D.X., Li, M., Qin, C., Wang, P., Savarese, S., Xiong, C., Joty, S.: A Survey of Frontiers in LLM Reasoning: Inference Scaling, Learning to Reason, and Agentic Systems (2025). <https://arxiv.org/abs/2504.09037>
- [43] Winston, C., Just, R.: A taxonomy of failures in tool-augmented llms. In: *AST 2025*, pp. 125–135 (2025). <https://doi.org/10.1109/AST66626.2025.00019>

- [44] Cemri, M., Pan, M.Z., Yang, S., Agrawal, L.A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J.E., Stoica, I.: Why Do Multi-Agent LLM Systems Fail? (2025)
- [45] Le Jeune, P., Liu, J., Rossi, L., Dora, M.: Realharm: A collection of real-world language model application failures. In: LLMSEC 2025, pp. 87–100 (2025)
- [46] Ronanki, K., Cabrero-Daniel, B., Berger, C.: Prompt Smells: An Omen for Undesirable Generative AI Outputs (2024). <https://arxiv.org/abs/2401.12611>
- [47] Agrawal, A., Kedia, N., Agarwal, A., Mohan, J., Kwatra, N., Kundu, S., Ramjee, R., Tumanov, A.: On Evaluating Performance of LLM Inference Serving Systems (2025)
- [48] Zhuo, T.Y., He, J., Sun, J., Xing, Z., Lo, D., Grundy, J., Du, X.: Identifying and Mitigating API Misuse in Large Language Models (2025)
- [49] Esfahani, A.M., Kahani, N., Ajila, S.A.: Understanding Defects in Generated Codes by Language Models (2024). <https://arxiv.org/abs/2408.13372>
- [50] Diaz-De-Arcaya, J., López-De-Armentia, J., Miñón, R., Ojanguren, I.L., Torre-Bastida, A.I.: Large language model operations (llmops): Definition, challenges, and lifecycle management. In: 2024 9th International Conference on Smart and Sustainable Technologies (SpliTech), pp. 1–4 (2024). <https://doi.org/10.23919/SpliTech61897.2024.10612341>
- [51] Tantithamthavorn, C.K., Palomba, F., Khomh, F., Chua, J.J.: Mlops, llmops, fmops, and beyond. *IEEE Software* **42**(1), 26–32 (2025) <https://doi.org/10.1109/MS.2024.3477014>
- [52] Google Cloud: What is LLMOps (large language model operations)? (2026). <https://cloud.google.com/discover/what-is-llmops>
- [53] IBM: What is LLMOps? Accessed: March 20, 2026 (2026). <https://www.ibm.com/think/topics/llmops>

11 Appendix

Selected Papers

- [SP54] Liu, M.X., Liu, F., Fiannaca, A.J., Koo, T., Dixon, L., Terry, M., Cai, C.J.: We need structured output: Towards user-centered constraints on large language model output. (2024). <https://doi.org/10.1145/3613905.3650756>
- [SP55] Papaioannou, K., Doudali, D., Thaleia, K.: The importance of workload choice in evaluating llm inference systems. In: Proceedings of the ACM

Conference on Systems and Machine Learning. Association for Computing Machinery, ??? (2024). <https://doi.org/10.1145/3642970.3655823>

- [SP56] Papaioannou, K., Doudali, D., Thaleia, K.: Improving the efficiency of llm inference serving systems. In: Lecture Notes in Computer Science. Springer, ??? (2025). https://doi.org/10.1007/978-3-031-90203-1_39
- [SP57] Palla, D., Slaby, A.: Evaluation of generative ai models in python code generation: A comparative study. IEEE Access **0**(0), 1–14 (2025) <https://doi.org/10.1109/ACCESS.2025.3560244>
- [SP58] Sakai, K., Uehara, Y., Kashihara, S.: Implementation and evaluation of llm-based conversational systems on a low-cost device. In: Proceedings of the 2024 IEEE Global Humanitarian Technology Conference (GHTC), pp. 1–8. IEEE, ??? (2024). <https://doi.org/10.1109/GHTC62424.2024.10771565>
- [SP59] Liu, M.X., Liu, F., Fiannaca, A.J., Koo, T., Dixon, L., Terry, M., Cai, C.J.: “We Need Structured Output”: Towards User-centered Constraints on Large Language Model Output. In: Extended Abstracts of the CHI Conference on Human Factors in Computing Systems. CHI EA '24. Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3613905.3650756> . <https://doi.org/10.1145/3613905.3650756>
- [SP60] Kharitonov, D.: Enforcing JSON Outputs in Commercial LLMs. Towards Data Science. <https://medium.com/data-science/enforcing-json-outputs-in-commercial-llms-3db590b9b3c8> Accessed 2025-09-25
- [SP61] Developer Service: A Practical Guide on Structuring LLM Outputs with Pydantic. <https://dev.to/devasservice/a-practical-guide-on-structuring-llm-outputs-with-pydantic-50b4> Accessed 2025-09-25
- [SP62] Wyman, M., Barber, S.: How to Validate the Output of LLM-Based Products. <https://okareo.com/blog/posts/validate-llm-output> Accessed 2025-09-25
- [SP63] Modelmetry: How To Ensure LLM Output Adheres to a JSON Schema. <https://modelmetry.com/blog/how-to-ensure-llm-output-adheres-to-a-json-schema> Accessed 2025-09-25
- [SP64] OpenAI: OpenAI Platform Documentation (2025). <https://platform.openai.com/docs> Accessed 2025-09-25
- [SP65] Microsoft Learn: Azure OpenAI - Documentation (Quotas, Structured Outputs, How-To) (2025). <https://learn.microsoft.com/en-us/azure/ai-foundry/openai/> Accessed 2025-09-25

- [SP66] OpenAI: API Reference - Structured model outputs (2025). <https://platform.openai.com/docs/guides/structured-outputs> Accessed 2025-09-25
- [SP67] mariafilippa: PydanticOutputParser has high chance failing when completion contains new line #3709. GitHub issue (2023). <https://github.com/hwchase17/langchain/issues/3709> Accessed 2025-09-25
- [SP68] arena-ai: Structured-logprobs. OS library: enhances OpenAI Structured Outputs with token logprobs. <https://github.com/arena-ai/structured-logprobs> Accessed 2025-09-25
- [SP69] GitHub Pull Request. streaming JSON missing required; completions logprobs structure. GitHub Pull Request. <https://github.com/dgy516/vllm-cibench/pull/60>
- [SP70] GitHub Issue. Structured output is not correctly enforced when using GPT-OSS. GitHub Issue. <https://github.com/vllm-project/vllm/issues/23120>
- [SP71] GitHub Issue. Python - OpenAI Responses client structured output does not work with streaming responses. GitHub Issue. <https://github.com/microsoft/agent-framework/issues/238>
- [SP72] GitHub Pull Request. Responses - add structured output for SDK. GitHub Pull Request. <https://github.com/BerriAI/litellm/pull/14206>
- [SP73] Karatas, E.: Structured output generation in llms: Json schema and grammar-based decoding (2025). Accessed 2025-12-29
- [SP74] Docherty, A.: Structured output in xml using langchain (2025). Accessed 2025-12-29
- [SP75] Kim, D.: Guided json with llms: From raw pdfs to structured intelligence (2025). Accessed 2025-12-29
- [SP76] Ndzomga, F.: Structured outputs from open source llms: Techniques and best practices (2024). Accessed 2025-12-29
- [SP77] Blackburn, P.: Exploring llm citation generation in 2025 (2025). Accessed 2025-12-29
- [SP78] gopenai: Mastering structured llm calls: Langchain finetuning guide (2025). Accessed 2025-12-29
- [SP79] Verma, K.: Structured (json) output via llms (2024). Accessed 2025-12-29
- [SP80] Jay, D.: Llm-based structured generation using jsonschema (2024). Accessed 2025-12-29

- [SP81] Desai, M.: Output formatting strategies: Getting exactly what you want, how ... (2025). Accessed 2025-12-29
- [SP82] mychen76: Practical techniques to constraint llm output in json format (2023). Accessed 2025-12-29
- [SP83] Mete: Control llm output with response type and schema (2024). Accessed 2025-12-29
- [SP84] dev.to: Crafting structured JSON responses: Ensuring consistent output ... (2024). Accessed 2025-12-29
- [SP85] Venkatrama, S.: Taming LLMs: How to Get Structured Output Every Time (Even for Big Responses). <https://dev.to/shrsv/taming-llms-how-to-get-structured-output-every-time-even-for-big-responses-445c> Accessed 2025-12-29
- [SP86] OpenAI: Using the Vision API: Best Practices - API - OpenAI Developer. <https://community.openai.com/t/using-the-vision-api-best-practices/942342> Accessed 2025-12-29
- [SP87] Pydantic: Add LLM structured output example using Pydantic AI (2025). <https://github.com/pydantic/pydantic/pull/12593> Accessed 2025-12-29
- [SP88] ksyed-data: Use Pydantic for Response Validation (2025). <https://github.com/ksyed-data/renovation-tracker-backend-2025/issues/19> Accessed 2025-12-29
- [SP89] pguso: New Example: Add validation for structured outputs from LLM (2025). <https://github.com/pguso/ai-agents-from-scratch/issues/4> Accessed 2025-12-29
- [SP90] mhattingpete: Structured Output Validation and Error Recovery (2025). <https://github.com/mhattingpete/receipt-scanner-app/issues/5> Accessed 2025-12-29
- [SP91] hev: Feature Request: Add schema validation check type (2025). <https://github.com/hev/vibecheck/issues/32> Accessed 2025-12-29
- [SP92] aplashard: Develop Chapter: LLM Schema Validation and Classification Tasks (2025). <https://github.com/aplashard/llm-evals-book/issues/22> Accessed 2025-12-29
- [SP93] marcosomma: Harden LLM output parsing and schema validation (2025). <https://github.com/marcosomma/orca-reasoning/issues/14> Accessed 2025-12-29
- [SP94] confident-ai: BAML for ensuring that the required output format is

- maintained (2025). <https://github.com/confident-ai/deepeval/issues/1610> Accessed 2025-12-29
- [SP95] BAMresearch: Integrate JSON structure to guide LLM in data creation (2025). <https://github.com/BAMresearch/chatBIS/issues/6> Accessed 2025-12-29
- [SP96] TandoorRecipes: Add AI schema models for structured output and response validation (2025). <https://github.com/TandoorRecipes/recipes/pull/4289> Accessed 2025-12-29
- [SP97] arakoodev: JSON LLM output formatting (2025). <https://github.com/arakoodev/EdgeChains/issues/119> Accessed 2025-12-29
- [SP98] gsdindlinger: Output Formatting (2025). <https://github.com/gsdindlinger/TabTree-Table-QA-on-Full-Documents/issues/19> Accessed 2025-12-29
- [SP99] DavidAguilarParedes: [Bug] Ragas Dataset Generation Fails with Invalid JSON when using Large Knowledge Graphs on Local LLM (2025). <https://github.com/DavidAguilarParedes/mia-uc-chile-proyecto-aplicado-1/issues/2> Accessed 2025-12-29
- [SP100] Lee0110: Add type field to message structure for extensibility (2025). <https://github.com/Lee0110/yl-IM-flutter/pull/1> Accessed 2025-12-29
- [SP101] 567-labs: Instructor: Structured Outputs for LLMs. <https://github.com/567-labs/instructor> Accessed 2025-12-29
- [SP102] Jkrunal7: Structured Output Generation With AI. https://github.com/Jkrunal7/Structured_Output_Generation_with_AI Accessed 2025-12-29
- [SP103] v-checha: V-checha/valai. <https://github.com/v-checha/valai> Accessed 2025-12-29
- [SP104] mt7180: mt7180/llm-validation (2025). <https://github.com/mt7180/llm-validation> Accessed 2025-12-29
- [SP105] vivekvjnk: Heimdall. <https://github.com/vivekvjnk/Heimdall> Accessed 2025-12-29
- [SP106] jhd3197: Prompture. <https://github.com/jhd3197/Prompture> Accessed 2025-12-29
- [SP107] dani2112: dani2112/guardrails-guidance-tutorial (2025). <https://github.com/dani2112/guardrails-guidance-tutorial> Accessed 2025-12-29
- [SP108] petrukha-ivan: Petrukha-ivan/mlx-swift-structured. <https://github.com/petrukha-ivan/mlx-swift-structured> Accessed 2025-12-29

- [SP109] emre-karatas: BNF-Constrained Generation for LLMs. <https://github.com/emre-karatas/bnf-constrained-generation> Accessed 2025-12-29
- [SP110] deepankarm: Deepankarm/godantic. <https://github.com/deepankarm/godantic> Accessed 2025-12-29
- [SP111] Kwun-Morfitt, O.: Parsec. <https://github.com/olliem/parsec> Accessed 2025-12-29
- [SP112] yogeshkukreti: yogeshkukreti/langchain-structured-output (2025). <https://github.com/yogeshkukreti/langchain-structured-output> Accessed 2025-12-29
- [SP113] 567-labs: Instructor-go: Structured LLM Outputs. <https://github.com/567-labs/instructor-go> Accessed 2025-12-29
- [SP114] kapilreddy: Instructor-clj. <https://github.com/kapilreddy/instructor-clj> Accessed 2025-12-29
- [SP115] kishoretvk: jsonAI. <https://github.com/kishoretvk/jsonAI> Accessed 2025-12-29
- [SP116] Morishige, M., Koshihara, R.: Ensuring reproducibility in generative ai systems for general use cases: A framework for regression testing and open datasets (2025) <https://doi.org/10.48550/arXiv.2505.02854>
- [SP117] Xiao, B., Kantarci, B., Kang, J., Niyato, D., Guizani, M.: Efficient prompting for llm-based generative internet of things. *IEEE Internet of Things Journal* **0**(0), 1–14 (2024) <https://doi.org/10.1109/JIOT.2024.3470210>
- [SP118] Buchicchio, E., De Angelis, A., Moschitta, A., Santoni, F., S. Marco, L., Carbone, P.: Design, validation, and risk assessment of llm-based generative ai systems operating in the legal sector. In: *Proceedings of the 2024 IEEE International Symposium on Systems Engineering (ISSE)*, pp. 1–8. IEEE, ??? (2024). <https://doi.org/10.1109/ISSE63315.2024.10741134> . <https://ieeexplore.ieee.org/document/10741134>
- [SP119] Author, F., Author, S., Author, T.: Themes of building llm-based applications for production: A practitioner’s view. In: *Proceedings of the 2025 IEEE/ACM 4th International Conference on AI Engineering (CAIN)*. IEEE, Ottawa, Canada (2025)
- [SP120] Han, T., Wang, Z., Fang, C., Zhao, S., Ma, S., Chen, Z.: Token-budget-aware llm reasoning. In: *ACL 2025*, pp. 24842–24855. Association for Computational Linguistics, ??? (2025). <https://doi.org/10.18653/v1/2025.findings-acl.1274> . <https://aclanthology.org/2025.findings-acl.1274/>

- [SP121] Chen, Z., Ye, Y., Zhou, Z.: Adaptively Robust LLM Inference Optimization under Prediction Uncertainty (2025). <https://arxiv.org/abs/2508.14544>
- [SP122] Anthropic: Claude Documentation (API, Models) (2025). <https://docs.claude.com/> Accessed 2025-09-25
- [SP123] Google: Gemini API - Google AI for Developers (2025). <https://ai.google.dev/> Accessed 2025-09-25
- [SP124] Google Cloud: Quotas and limits - BigQuery (2025). <https://cloud.google.com/bigquery/quotas> Accessed 2025-09-25
- [SP125] Amazon Web Services: Timeouts, retries and backoff. <https://aws.amazon.com/fr/builders-library/timeouts-retries-and-backoff-with-jitter>. Accessed 2025-09-25 (2025)
- [SP126] OpenAI: openai-python. <https://github.com/openai/openai-python>. Accessed 2025-09-25 (2025)
- [SP127] GitHub Issue. LangChain expects at least one chunk from a streaming trace after timeout. GitHub Issue. <https://github.com/DataDog/dd-trace-py/issues/14688>
- [SP128] GitHub Issue. Streaming inactivity timeout incorrectly aborts after total timeout (@langchain/openai "1.0.0-alpha.1"). GitHub Issue. <https://github.com/langchain-ai/langchainjs/issues/9088>
- [SP129] GitHub Issue. Add production-ready Pipedrive CRM integration with deduplication and async sync. GitHub Pull Request. <https://github.com/jcartervi/MyYachtValue/pull/36>
- [SP130] GitHub Issue. BPMN Assistant API WAITING timeout when self-hosting on own instance. GitHub Issue. <https://github.com/jtlicardo/bpmn-assistant/issues/33>
- [SP131] GitHub Issue. Timeout. GitHub Issue. <https://github.com/eslarnov/ttmg-server/issues/19>
- [SP132] GitHub Pull Request. handle empty stream responses from GLM models. GitHub Pull Request. <https://github.com/RooCodeInc/Roo-Code/pull/8483>
- [SP133] Stack Overflow. "connection to OpenAI API failed with status: 400 error: -19577 is less than the minimum of 1 - max_tokens". Stack Overflow (accepted answer). <https://stackoverflow.com/questions/77172214/rag-error-connection-to-openai-api-failed-with-status-400-error-19577>
- [SP134] Stack Overflow. AI server crashes when I make a request

- after adding OpenAI code. Stack Overflow (accepted answer). <https://stackoverflow.com/questions/77354317/ai-server-crashes-when-i-make-a-request-after-adding-openai-code>
- [SP135] AI, N.: What are large language model settings: Temperature, top p and ... (2024). Accessed 2025-12-29
- [SP136] Malhar: Understanding llm settings for optimal performance (2024). Accessed 2025-12-29
- [SP137] VectorShift: Large language model settings: Temperature, top p and max tokens (2024). Accessed 2025-12-29
- [SP138] hev: Feature Request: Enhance llm_judge with configurable model and parameters (2025). <https://github.com/hev/vibecheck/issues/35> Accessed 2025-12-29
- [SP139] run-llama: [Feature Request] Uniform default LLM params across providers (2025). https://github.com/run-llama/llama_index/issues/19730 Accessed 2025-12-29
- [SP140] intel: default values of max_generated_tokens, top_k, top_p, and temperature? (2025). <https://github.com/intel/ipex-llm/issues/11033> Accessed 2025-12-29
- [SP141] Minh, N.N., Baker, A., Neo, C., Roush, A.G., Kirsch, A., Shwartz-Ziv, R.: Turning up the heat: Min-p sampling for creative and coherent llm outputs (2024)
- [SP142] Minh, N.N., Baker, A., Neo, C., Roush, A.G., Kirsch, A., Shwartz-Ziv, R.: Turning up the heat: Min-p sampling for creative and coherent llm outputs. In: ICLR 2025 (2025)
- [SP143] Montandon, J.E., Silva, L.L., Politowski, C., Prates, D., Brito Bonifácio, A., Boussaidi, G.E.: Unboxing default argument breaking changes in data science libraries (2025) <https://doi.org/10.48550/arXiv.2408.05129> arXiv:2408.05129 [cs.SE]. JSS
- [SP144] Vellum AI: LLM Temperature: How It Works and When You Should Use It (2025). <https://www.vellum.ai/llm-parameters/temperature> Accessed 2025-09-25
- [SP145] Hugging Face: Transformers Documentation (Generation, Chat Templates, Model Revisions) (2025). <https://huggingface.co/docs/transformers> Accessed 2025-09-25
- [SP146] Ollama: Modelfile: Valid parameters and values (2025). <https://github.com/ollama/ollama/blob/main/docs/modelfile.md#valid-parameters-and-values> Accessed 2025-09-25

- [SP147] GitHub Issue/PR. MCP-USE with VLLM gpt-oss:20b via ChatOpenAI. GitHub Issue/PR. <https://github.com/vllm-project/vllm/issues/26806>
- [SP148] GitHub Issue/PR. When streaming responses with the OpenAI Responses API, temperature is not captured correctly. GitHub Issue/PR. <https://github.com/langfuse/langfuse/issues/9566>
- [SP149] Wang, K.: A comprehensive guide to llm temperature (2025). Accessed 2025-12-29
- [SP150] Pathak, T.: Set the temperature right: Optimizing temperature settings in prompt engineering (2024). Accessed 2025-12-29
- [SP151] kaviyadharishini21: Impact of temperature on large language models (llms) (2024). Accessed 2025-12-29
- [SP152] Balarabe, T.: Understanding llm temperature: Creativity vs. consistency (2025). Accessed 2025-12-29
- [SP153] u/Extreme-Wall9508: What Is the Right Temperature Parameter to Work With? : r/LLMDevs. https://www.reddit.com/r/LLMDevs/comments/1i3113d/what_is_the_right_temperature_parameter_to_work/ Accessed 2025-12-29
- [SP154] Herrmannova, D.: Temperature sampling (2024). Accessed 2025-12-29
- [SP155] Šubonis, M.: Zero temperature randomness in llms (2025). Accessed 2025-12-29
- [SP156] Patel, A.: "LLM Temperature" Is NOT the Little Slider You Think It Is. Ever ... https://www.linkedin.com/posts/ashishpatel2604_llm-temperature-is-not-the-little-slider-activity-7391720132559118336-2D6C Accessed 2025-12-29
- [SP157] Face, H.: The document of generation seems to wrongly describe the default value of top_p, top_k and temperature (2025). <https://github.com/huggingface/transformers/issues/35045> Accessed 2025-12-29
- [SP158] OpenHands: Bug: Azure GPT-5 fails due to hardcoded temperature in responses_options.py (2025). <https://github.com/OpenHands/software-agent-sdk/issues/986> Accessed 2025-12-29
- [SP159] micz: Expose model temperature control in ThunderAI connection settings (global default + optional per-prompt override) (2025). <https://github.com/micz/ThunderAI/issues/561> Accessed 2025-12-29
- [SP160] penguoir: Allow user-defined temperature (2025). https://github.com/penguoir/active_cortex/issues/3 Accessed 2025-12-29

- [SP161] Pepelespooder: Fix Darkmoon temperature handling to respect user values and prevent dirty state (2025). <https://github.com/Pepelespooder/BambuStudio/pull/13> Accessed 2025-12-29
- [SP162] haktancetin: LLM Randomness (2025). https://github.com/haktancetin/496_CookBuddyProject/issues/24 Accessed 2025-12-29
- [SP35] OpenAI Developer Community: Clarifications on Setting Temperature = 0. Discussion thread accessed 2025-12-09. <https://community.openai.com/t/clarifications-on-setting-temperature-0/886447>
- [SP164] Institute, P.E.: Complete Guide to Prompt Engineering with Temperature and Top-p. Accessed: 2025-12-31 (2024). <https://promptengineering.org/prompt-engineering-with-temperature-and-top-p/>
- [SP165] Reyes, F., Gamage, Y., Skoglund, G., Baudry, B., Monperrus, M.: Bump: A benchmark of reproducible breaking dependency updates. (2024). <https://doi.org/10.48550/arXiv.2401.09906> . <https://doi.org/10.48550/arXiv.2401.09906>
- [SP166] Venturini, D., Cogo, F.R., Polato, I., Gerosa, M.A., Wiese, I.S.: I depended on you and you broke me: An empirical study of manifesting breaking changes in client packages (2023) <https://doi.org/10.48550/arXiv.2301.04563>
- [SP167] Montandon, J.E., Silva, L.L., Politowski, C., Prates, D., Bonifacio, A.B., Boussaidi, G.E.: Unboxing default argument breaking changes in data science libraries (2024) <https://doi.org/10.48550/arXiv.2408.05129>
- [SP168] Silva Simões, I.R., Venson, E.: Evaluating source code quality with large language models: a comparative study, 103–113 (2024) <https://doi.org/10.1145/3701625.3701650>
- [SP169] Wilson, G., Aruliah, D.A., Brown, C.T., Chue Hong, N.P., Davis, M., Guy, R.T., Haddock, S.H.D., Huff, K.D., Mitchell, I.M., Plumbley, M.D., Waugh, B., White, E.P., Wilson, P.: Best practices for scientific computing. *PLOS Biology* **12**(1), 1001745 (2014)
- [SP170] Venturini, D., Cogo, F.R., Polato, I., Gerosa, M.A., Wiese, I.S.: I depended on you and you broke me: An empirical study of manifesting breaking changes in client packages (2023) <https://doi.org/10.48550/arXiv.2301.04563> arXiv:2301.04563. TOSEM, 2023
- [SP171] Reyes, F., Gamage, Y., Skoglund, G., Baudry, B., Monperrus, M.: Bump: A benchmark of reproducible breaking dependency updates. In: *SANER 2024* (2024). <https://doi.org/10.48550/arXiv.2401.09906> . <https://arxiv.org/abs/2401.09906> Accessed 2025-09-25

- [SP172] Morishige, M., Koshihara, R.: Ensuring reproducibility in generative ai systems for general use cases: A framework for regression testing and open datasets. (2025). <https://doi.org/10.48550/arXiv.2505.02854>
- [SP173] Microsoft Learn: Design to Support Foundation Model Life Cycles (2025). <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/manage-foundation-models-lifecycle> Accessed 2025-09-25
- [SP174] OpenRouter: OpenRouter.ai - One API for Any Model. <https://openrouter.ai/> Accessed 2025-09-25
- [SP175] Jeong, M., Cho, J., Khang, M., Jung, D., Hong, T.: System message generation for user preferences using open-source models (2025)
- [SP176] Neumann, A., Kirsten, E., Zafar, M.B., Singh, J.: Position is power: System prompts as a mechanism of bias in large language models. (2025). <https://doi.org/10.1145/3715275.3732038> . <https://doi.org/10.1145/3715275.3732038>
- [SP177] Jeong, M., Cho, J., Khang, M., Jung, D., Hong, T.: System Message Generation for User Preferences using Open-Source Models (2025). <https://arxiv.org/abs/2502.11330>
- [SP178] Neumann, A., Kirsten, E., Zafar, M.B., Singh, J.: Position is power: System prompts as a mechanism of bias in large language models (llms). In: Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency. FAccT '25, pp. 573–598. ACM, ??? (2025). <https://doi.org/10.1145/3715275.3732038> . <http://dx.doi.org/10.1145/3715275.3732038>
- [SP179] Cleary, D.: System Messages: Best Practices, Real-world Experiments & Prompt Injection Protectors. PromptHub Blog. <https://www.prompthub.us/blog/everything-system-messages-how-to-use-them-real-world-experiments-prompt-injection-protectors> Accessed 2025-09-25
- [SP180] cyz3a5c0v1: What is the use case of System role (2023). <https://stackoverflow.com/questions/76272624/what-is-the-use-case-of-system-role> Accessed 2025-09-25
- [SP181] OpenAI: What Exactly Does a System Msg Do? <https://community.openai.com/t/what-exactly-does-a-system-msg-do/459409> Accessed 2025-12-29
- [SP182] Community, O.: Question Around the Structure of System, User, Assistant - API. <https://community.openai.com/t/question-around-the-structure-of-system-user-assistant/581078> Accessed 2025-12-29
- [SP183] OpenAI: Understanding Role Management in OpenAI's API:

- Two Methods Compared. <https://community.openai.com/t/understanding-role-management-in-openais-api-two-methods-compared/253289> Accessed 2025-12-29
- [SP184] OpenAI: The “system” Role - How It Influences the Chat Behavior - API. <https://community.openai.com/t/the-system-role-how-it-influences-the-chat-behavior/87353> Accessed 2025-12-29
- [SP185] OpenAI: Providing Context to the Chat API Before a Conversation - Prompting ... <https://community.openai.com/t/providing-context-to-the-chat-api-before-a-conversation/195853> Accessed 2025-12-29
- [SP186] dan_43009: The difference between system messages and user messages in ... (2024). Accessed 2025-12-29
- [SP187] Hakim, M.: Mastering prompt engineering: A guide to system, user, and ... (2025). Accessed 2025-12-29
- [SP188] u/thexdroid: What Is the Correct Way to Use System Message Roles? : R/ollama. https://www.reddit.com/r/ollama/comments/1dgw4w9/what_is_the_correct_way_to_use_system_message/ Accessed 2025-12-29
- [SP189] lgabs: use System, Human and AI patterns for prompts roles for OpenAI chat models (2025). <https://github.com/lgabs/dialog/issues/16> Accessed 2025-12-29
- [SP190] Wen, H., Wu, X., Sun, Y., Zhang, F., Chen, L., Wang, J., Liu, Y., Liu, Y., Zhang, Y., Li, Y.: Budgetthinker: Empowering budget-aware llm reasoning with control tokens (2025) <https://doi.org/10.48550/arXiv.2508.17196>
- [SP191] Wang, J., Jain, S., Zhang, D., Ray, B., Kumar, V., Athiwaratkun, B.: Reasoning in token economies: Budget-aware evaluation of llm reasoning strategies. (2024). <https://doi.org/10.18653/v1/2024.emnlp-main.1112> . <https://aclanthology.org/2024.emnlp-main.1112/>
- [SP192] Chen, Z., Ye, Y., Zhou, Z.: Adaptively robust llm inference optimization under prediction uncertainty (2025)
- [SP193] Han, T., Wang, Z., Fang, C., Zhao, S., Ma, S., Chen, Z.: Token-budget-aware llm reasoning. (2025). <https://doi.org/10.18653/v1/2025.findings-acl> . <https://doi.org/10.18653/v1/2025.findings-acl>
- [SP194] Lee, J., Shin, W., Yang, S., Song, K.-U., Lim, D., Kim, J., Kim, T.-H., Kim, B.-K.: Ergo: Efficient high-resolution visual understanding for vision-language models (2025) <https://doi.org/10.48550/arXiv.2509.21991>

- [SP195] Vasu, P.K.A., Faghri, F., Li, C.-L., Koc, C., True, N., Antony, A., Santhanam, G., Gabriel, J., Gräsch, P., Tuzel, O., Pouransari, H.: Fastvlm: Efficient vision encoding for vision language models (2024) <https://doi.org/10.48550/arXiv.2412.13303>
- [SP196] Qian, J., Wang, C., Yang, Y., Zhang, C., Jiang, H., Luo, X., Kang, Y., Lin, Q., Zhang, A., Jiang, S., Cao, T., Mao, T., Banerjee, S., Liu, G., Rajmohan, S., Zhang, D., Yang, Y., Zhang, Q., Qiu, L.: Zoomer: Adaptive image focus optimization for black-box mllm (2025) <https://doi.org/10.48550/arXiv.2505.00742>
- [SP197] Microsoft Corporation: Azure OpenAI in Microsoft Foundry Models REST API. Accessed 2025-12-09. <https://learn.microsoft.com/en-us/azure/ai-foundry/openai/latest?view=foundry-classic>
- [SP198] Anthropic: Create a Text Completion Claude API Reference. Accessed 2025-12-09. https://docs.anthropic.com/claude/reference/complete_post
- [SP199] Willison, S.: Llm-anthropic. Python package version 0.23 accessed 2025-12-09. <https://pypi.org/project/llm-anthropic/>
- [SP200] n8n.io: AI Agent with Anthropic Models Fails Temperature and Top_p Cannot Both Be Specified for this Model. GitHub issue accessed 2025-12-09. <https://github.com/n8n-io/n8n/issues/18304>
- [SP201] Medium: Setting top-k, top-p and temperature in llms (2024). Accessed 2025-12-29
- [SP202] Albert: Large language model settings: Temperature, top p and max ... (2024). Accessed 2025-12-29
- [SP203] RepoWise: Ensure Deterministic & Fully Consistent LLM Output (2025). <https://github.com/RepoWise/backend/issues/11> Accessed 2025-12-29
- [SP204] mpfaffenberger: Feature Request: Add Recommended Sampling Parameters for GLM Model (2025). https://github.com/mpfaffenberger/code_puppy/issues/70 Accessed 2025-12-29
- [SP205] OpenAI: Safety best practices. <https://platform.openai.com/docs/guides/safety-best-practices>. Accessed 2025-12-10 (2025)
- [SP206] Google DeepMind: Safety guidance. <https://ai.google.dev/gemini-api/docs/safety-guidance>. Accessed 2025-12-10 (2025)
- [SP207] Anthropic: Create a Message. <https://platform.claude.com/docs/en/api/python/messages/create>. Accessed 2025-12-10 (2025)

- [SP208] raz-alon, contributors: [Bug]: Anthropic API throws Bad Request when user.-id in metadata contains email or phone number. <https://github.com/BerriAI/litellm/issues/10106>. GitHub issue 10106, opened 2025-04-17, accessed 2025-12-10 (2025)
- [SP209] OpenAI: Need Help: Facing OpenAI Usage Violation Due to User's Abuse - API. <https://community.openai.com/t/need-help-facing-openai-usage-violation-due-to-users-abuse/1004947> Accessed 2025-12-29
- [SP210] Community, O.D.: API Ban from User Abuse - API - OpenAI Developer Community. <https://community.openai.com/t/api-ban-from-user-abuse/25400> Accessed 2025-12-29
- [SP211] Community, O.: Any Suggestions for Preventing Openai API Abuse - API - OpenAI ... <https://community.openai.com/t/any-suggestions-for-preventing-openai-api-abuse/1075315> Accessed 2025-12-29
- [SP212] OpenAI: Lessons Learned on Language Model Safety and Misuse. <https://openai.com/index/language-model-safety-and-misuse/> Accessed 2025-12-29
- [SP213] OpenAI: API Policy Violation Warning - OpenAI Developer Community. <https://community.openai.com/t/api-policy-violation-warning-advice-on-how-to-best-resolve/788354> Accessed 2025-12-29
- [SP214] Anthropic: Detecting and Countering Malicious Uses of Claude. <https://www.anthropic.com/news/detecting-and-countering-malicious-uses-of-claude-march-2025> Accessed 2025-12-29
- [SP215] Tech, N.F.: Safeguarding your ai: Best practices for securing your openai api (2023). Accessed 2025-12-29
- [SP216] Oliveira, A.C., Azevedo, J.P.C., Ruback, L., Moreira, R., Teixeira, S.S., Teles, A.S.: Effect of explainable artificial intelligence on trust of mental health professionals in an ai-based system for suicide prevention. IEEE Access, 1–14 (2025) <https://doi.org/10.1109/ACCESS.2025.3556245>
- [SP217] Ijas, A.H., Jo, A.A., Raj, E.D.: Exploring explainable ai in large language models: Enhancing transparency and trust. In: 2024 11th International Conference on Advances in Computing and Communications (ICACC), Kochi, India, pp. 1–7 (2024). <https://doi.org/10.1109/ICACC63692.2024.10845370> . IEEE. Held 6–8 November 2024
- [SP218] Na, S., Jeong, G., Ahn, B.H., Young, J., Krishna, T., Kim, H.: Understanding

- performance implications of llm inference on cpus. In: Proceedings of the 2024 IEEE International Symposium on Workload Characterization (IISWC), pp. 1–12. IEEE, ??? (2024). <https://doi.org/10.1109/IISWC63097.2024.00024> . <https://ieeexplore.ieee.org/document/10763564>
- [SP219] Karthikeyan: Openai api security: Managing ai risk in chatbots (2025). Accessed 2025-12-29
- [SP220] ollama: Ollama/ollama. <https://github.com/ollama/ollama> Accessed 2025-12-29
- [SP221] owainlewis: owainlewis/awesome-artificial-intelligence (2025). <https://github.com/owainlewis/awesome-artificial-intelligence> Accessed 2025-12-29
- [SP222] humanlayer: 12-Factor Agents - Principles for building reliable LLM applications (2025). <https://github.com/humanlayer/12-factor-agents> Accessed 2025-12-29
- [SP223] DeepSeek-AI: DeepSeek-R1. <https://github.com/deepseek-ai/DeepSeek-R1> Accessed 2025-12-29
- [SP224] OpenAI: GPT 4.1 Character Encoding Issues? - Bugs - OpenAI Developer. <https://community.openai.com/t/gpt-4-1-character-encoding-issues/1236017> Accessed 2025-12-29
- [SP225] OpenAI: How to Pass Conversation History Back to the API - API - OpenAI. <https://community.openai.com/t/how-to-pass-conversation-history-back-to-the-api/697083> Accessed 2025-12-29
- [SP226] quic: LLM Output Text formatting Issue (2025). <https://github.com/quic/cloud-ai-sdk/issues/6> Accessed 2025-12-29
- [SP227] alltuner: Add error handling for LLM response parsing (2025). <https://github.com/alltuner/blogtuner/issues/43> Accessed 2025-12-29
- [SP228] Skyvern-AI: always capture llm artifacts (2025). <https://github.com/Skyvern-AI/skyvern/pull/4284> Accessed 2025-12-29
- [SP229] jerseycheese: Advanced provider settings and generation parameters (2025). <https://github.com/jerseycheese/Narrator/issues/899> Accessed 2025-12-29
- [SP230] docling-project: VLM pipeline hangs indefinitely during document processing with Transformers backend on NVIDIA GPU (2025). <https://github.com/docling-project/docling/issues/2472> Accessed 2025-12-29
- [SP231] Blaizzy: Command A Vision responses contain only "PAD;" tokens (2025). <https://github.com/Blaizzy/mlx-vlm/issues/487> Accessed 2025-12-29

- [SP232] ai-dynamo: feat: generalize VLM embedding extraction (2025). <https://github.com/ai-dynamo/dynamo/pull/1388> Accessed 2025-12-29
- [SP233] sgl-project: docs: Add token-in-token-out examples for LLM and VLM engines (2025). <https://github.com/sgl-project/sglang/pull/7262> Accessed 2025-12-29
- [SP234] M5Stack: Sampling parameters (temperature, top_k, top_p, etc.) not applied in LLM module (standalone mode) (2025). <https://github.com/m5stack/StackFlow/issues/22> Accessed 2025-12-29
- [SP235] mistralai: Feature request: Notify user when LLM response lacks content body (2025). <https://github.com/mistralai/mistral-vibe/issues/130> Accessed 2025-12-29
- [SP236] HouseOfBetterAuth: Implement conversation history/context for chat LLM calls (2025). <https://github.com/HouseOfBetterAuth/nuxt-better-auth-saas/issues/1> Accessed 2025-12-29
- [SP237] ollama: Ollama freezes when specifying chat roles for some models. (2025). <https://github.com/ollama/ollama/issues/7003> Accessed 2025-12-29
- [SP238] spring-projects: Chat memory advisor causes message ordering problem with default system message prompt (2025). <https://github.com/spring-projects/spring-ai/issues/4170> Accessed 2025-12-29
- [SP239] alexyang0826: Inference-Time Techniques for LLM Reasoning (2025). https://github.com/alexyang0826/ADSL_Sumer_School_2025/issues/6 Accessed 2025-12-29
- [SP240] Praveen76: LLMs-API-Usage-Best-Practices (2025). <https://github.com/Praveen76/LLMs-API-Usage-Best-Practices> Accessed 2025-12-29
- [SP241] feibaoBob: LLM-based_CAMD. https://github.com/feibaoBob/LLM-based_CAMD Accessed 2025-12-29
- [SP242] andrewyng: Aisuite. <https://github.com/andrewyng/aisuite> Accessed 2025-12-29
- [SP243] 0x6f677548: Unicode Injection Proof of Concept (2025). <https://github.com/0x6f677548/unicode-injection> Accessed 2025-12-29
- [SP244] DrPwner: PromptSniffer. <https://github.com/DrPwner/PromptSniffer> Accessed 2025-12-29
- [SP245] pablo-chacon: Adversarial LLM Threats (2025). <https://github.com/pablo-chacon/Adversarial-LLM-Threats> Accessed 2025-12-29

- [SP246] ridpath: LLM Vulnerability Scanner. <https://github.com/ridpath/llm-vuln-scanner> Accessed 2025-12-29
- [SP247] grafbase: Grafbase/nexus. <https://github.com/grafbase/nexus> Accessed 2025-12-29
- [SP248] Libr-AI: Libr-AI/OpenRedTeaming (2025). <https://github.com/Libr-AI/OpenRedTeaming> Accessed 2025-12-29
- [SP249] HqWu-HITCS: Awesome-LLM-Survey (2025). <https://github.com/HqWu-HITCS/Awesome-LLM-Survey> Accessed 2025-12-29
- [SP250] Gong, Y., Ran, D., Liu, J., Wang, C., Cong, T., Wang, A., Duan, S., Wang, X.: FigStep: Jailbreaking Large Vision-language Models Via Typographic Visual Prompts. <https://github.com/CryptoAILab/FigStep> Accessed 2025-12-29
- [SP251] 0xAIDR: AIDR Bastion. <https://github.com/0xAIDR/AIDR-Bastion> Accessed 2025-12-29